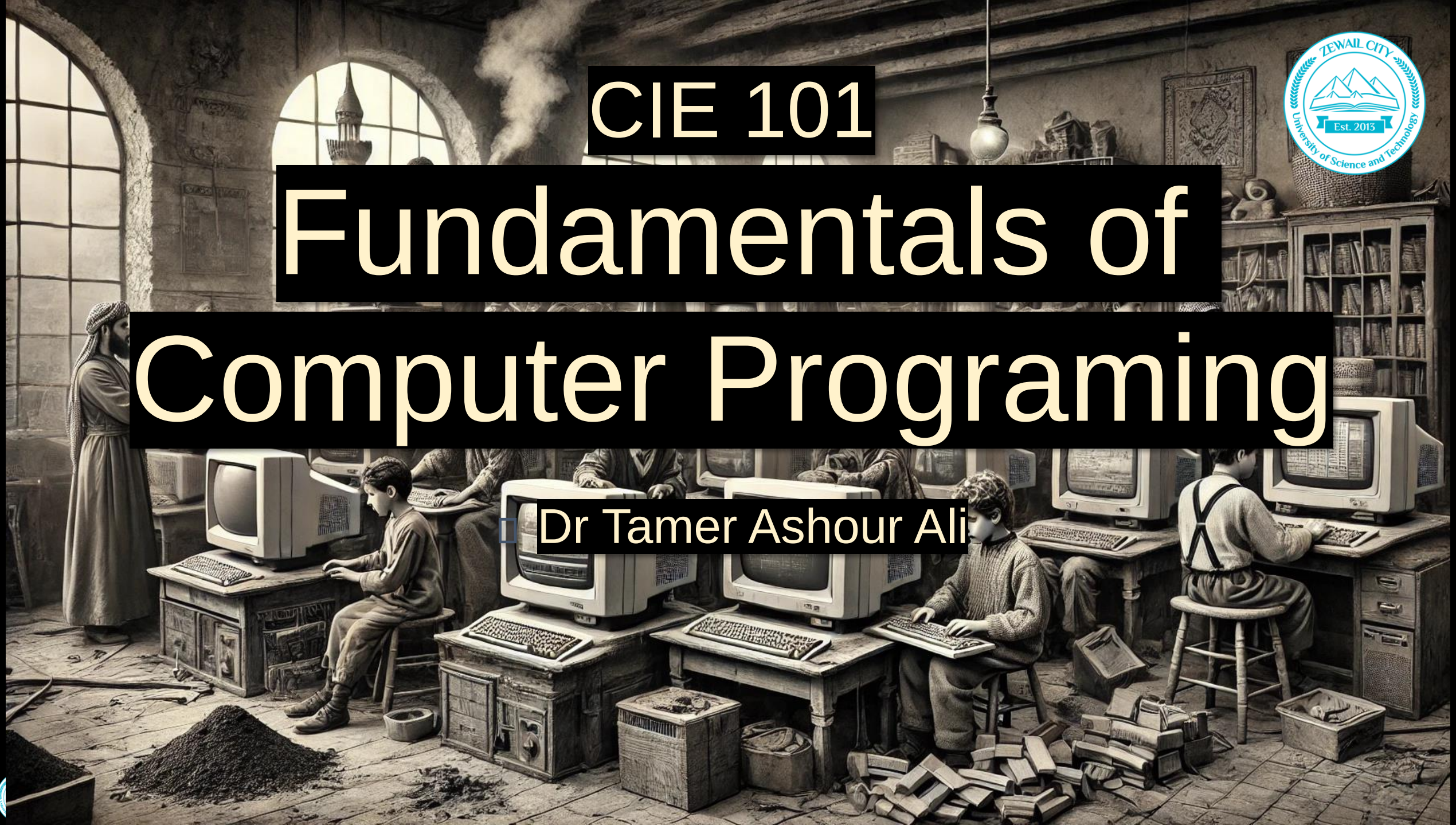
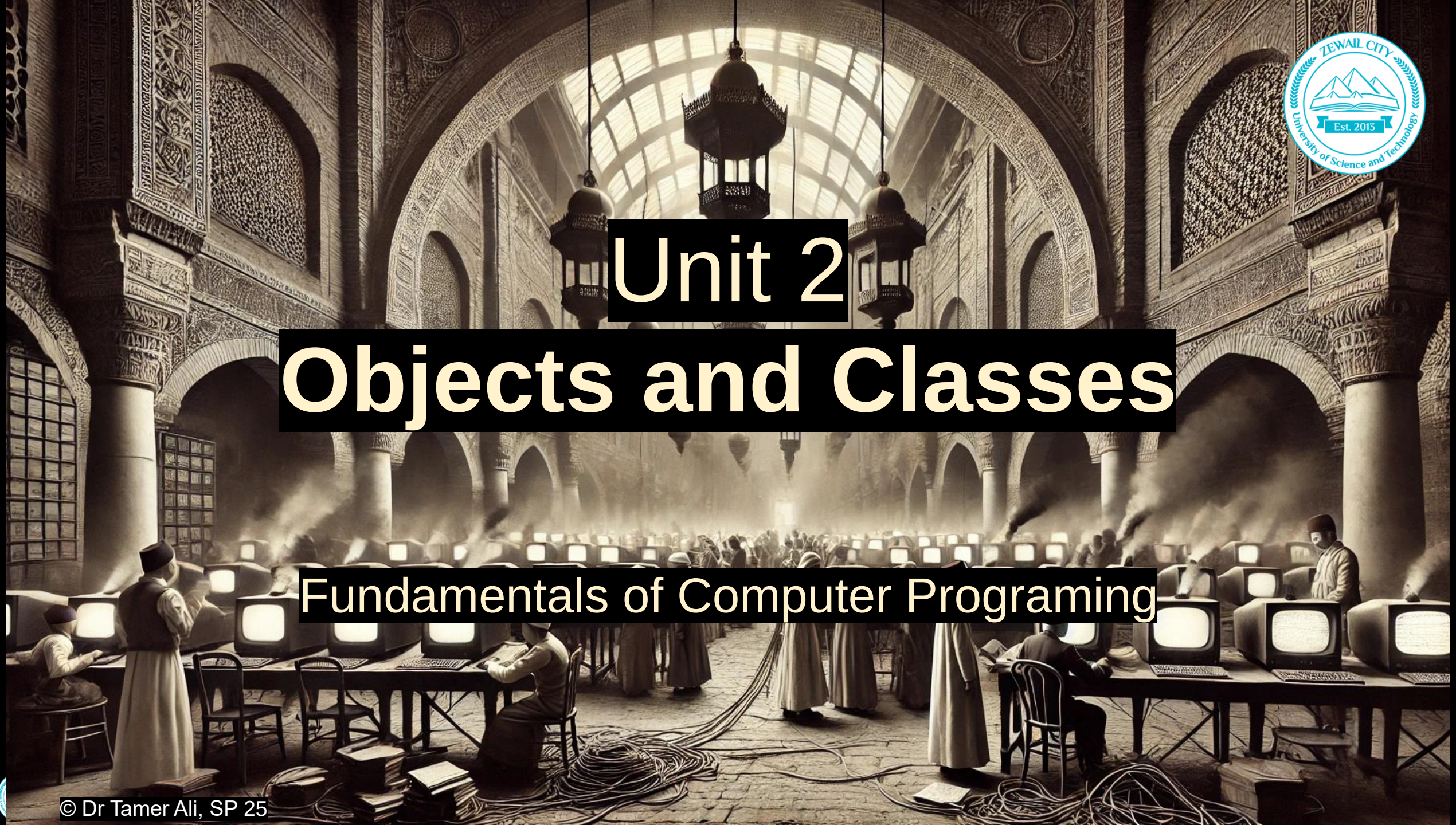

CIE 101
Fundamentals of
Computer Programing
Dr Tamer Ashour Ali
ZEWAIL CITY
Est. 2013
University of Science and Technology



Unit 2

Objects and Classes

Fundamentals of Computer Programing

Reference for these slides

- W. Savitch, Problem Solving with C++, Pearson
- Y. Liang, Introduction to Programming with C++. Pearson Education India, 2011.
- T. Gaddis, P. Sengupta, Starting out with C++: from control structures through objects. Pearson, 2012.
- M. Weisfeld, The Object-Oriented Thought Process. Pearson Education, 2009

Agenda

- Moving to Object Oriented Programing
- How to Declare a Class?
- Abstraction and Encapsulation
- Constant and Static Members
- Constructors and Destructors
- Initializing Member Data Fields
- Objects with Functions and Arrays
- Introduction to Designing an Object-Oriented Solution

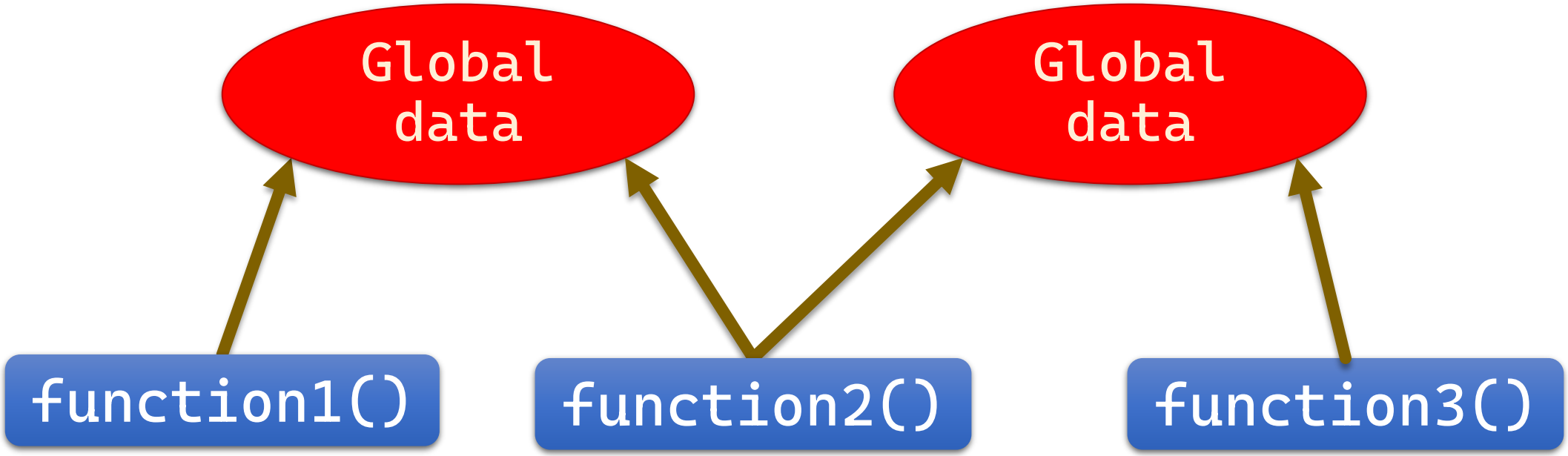
Moving to Object Oriented Programing

Procedural Programing

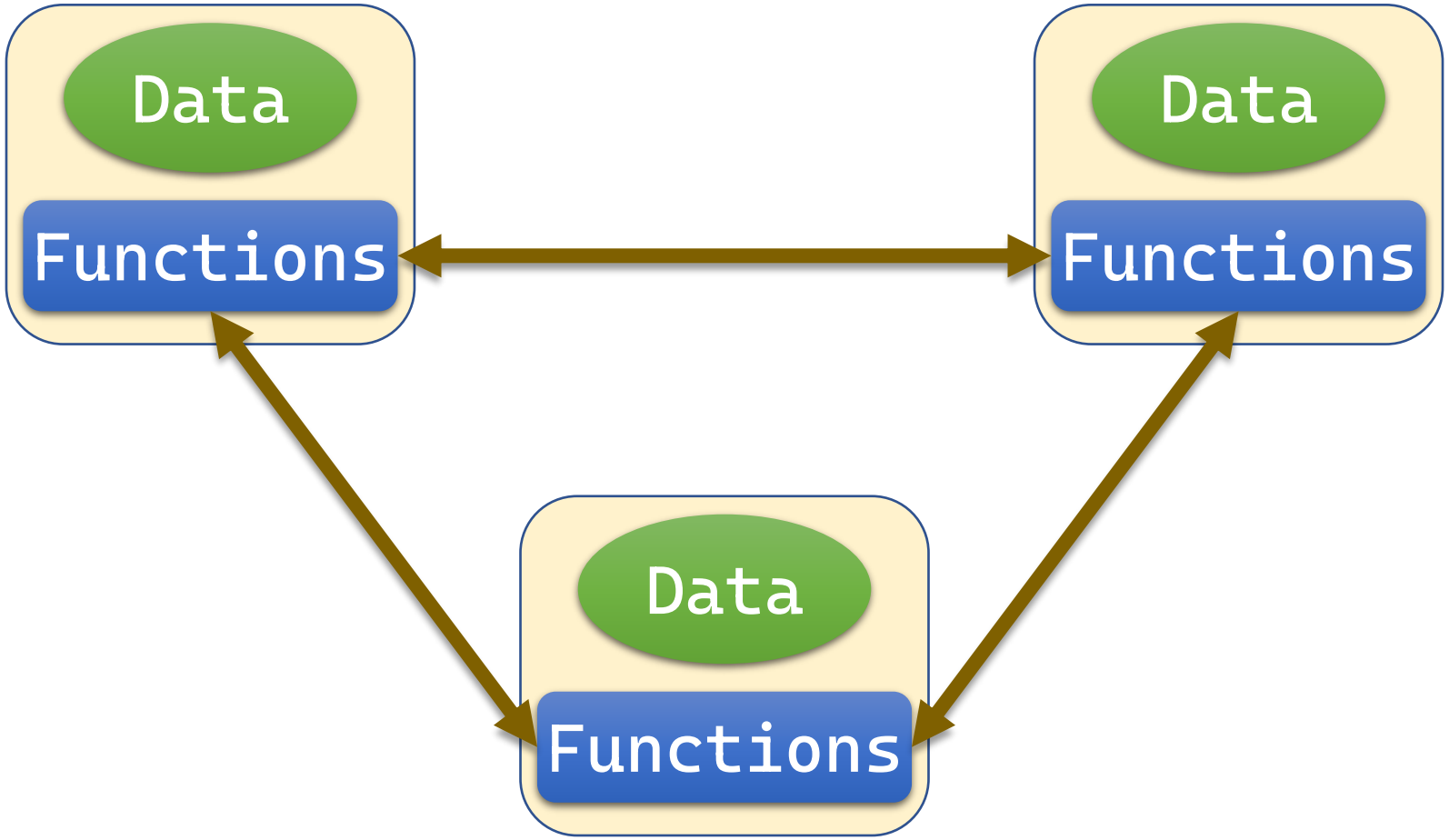
```
// Bank account data
double balance = 0.0;

void withdraw(double amount)
{
  if (amount > balance)
    cout << "Insufficient funds!"
         << endl;
  else
  {
    balance -= amount;
    cout << "New Balance: $"
         << balance << endl;
  }
}
```

```
void deposit(double amount)
{
  balance += amount;
  cout << "New Balance: $"
       << balance << endl;
}

int main()
{
  deposit(100);
  withdraw(30);
  withdraw(80);
}
```

Procedural Programing



Object Oriented Programing



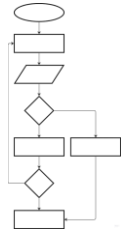
Object Oriented Programing

```
class BankAccount {
private: double balance;
public:
  BankAccount(double val)
  { balance = val; }

  void withdraw(double amount) {
    if (amount > balance)
      cout << "Insufficient funds!"
           << endl;
    else {
      balance -= amount;
      cout << "New Balance: $"
           << balance << endl;
    }
  }
```

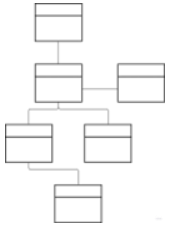
```
  void deposit(double amount) {
    balance += amount;
    cout << "New Balance: $"
         << balance << endl;
  }
};

int main() {
  BankAccount myAccount(50);
  myAccount.deposit(100);
  myAccount.withdraw(30);
  myAccount.withdraw(80);
}
```

Procedural Programing and Object-Oriented Programing

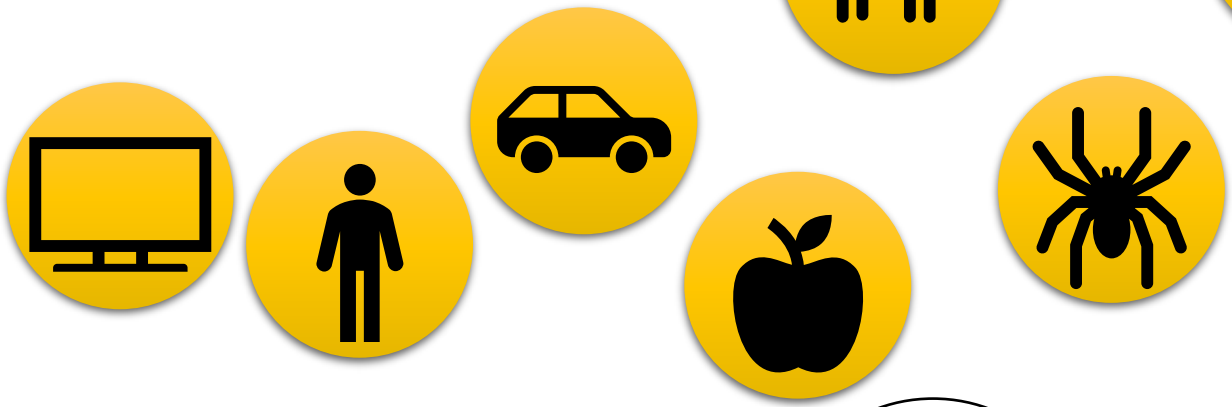
- Procedural programming focuses on the process/actions that occur in a program.
  - Tells the computer what to do step by step



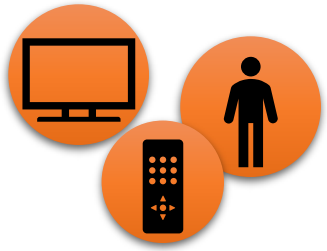
- Object-Oriented programming (OOP) is based on the data and the functions that operate on it. Objects are instances of ADTs that represent the data and its functions.
  - All computations are carried through OBJECTS
  - OBJECT: An element that knows how to act and how to interact with other elements



Object-Oriented Programming

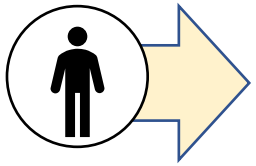


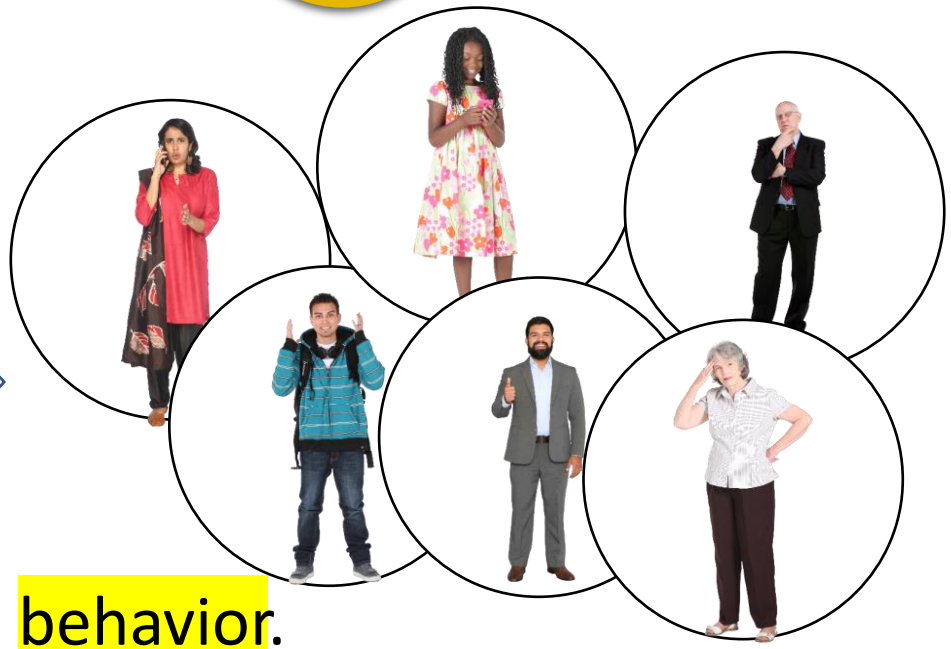
- Everything is an object.



- Objects can
  - act/interact
  - have their own memory

- Every object is an instance of a class
  - The class holds the shared behavior for its instances.

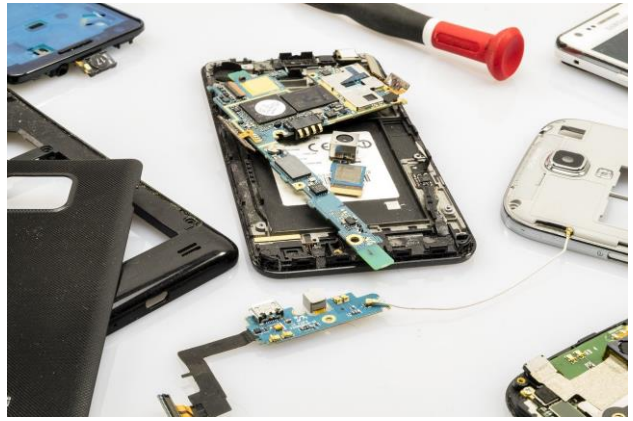


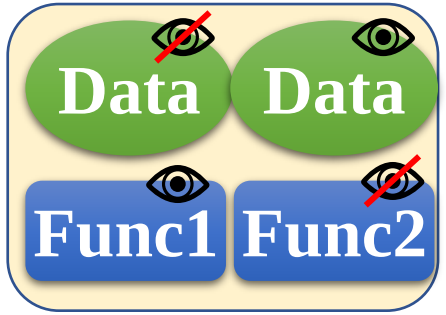


- Every object has a unique identity, state, and behavior.

Object-Oriented Programming

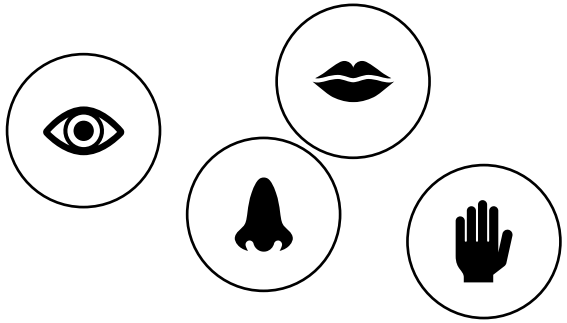
- Data hiding:
  - restricting access to certain members of an object
- Public interface:
  - members that are accessible outside of the object
- Allows the object to
  - provide access to some data and functions without sharing its internal details and design
  - provide some protection from data corruption



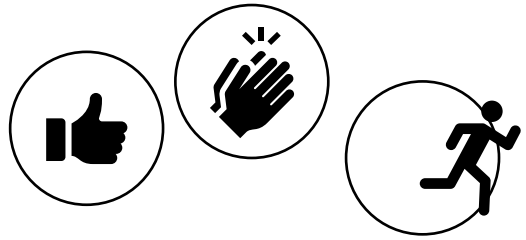


Object-Oriented Programming Terminology

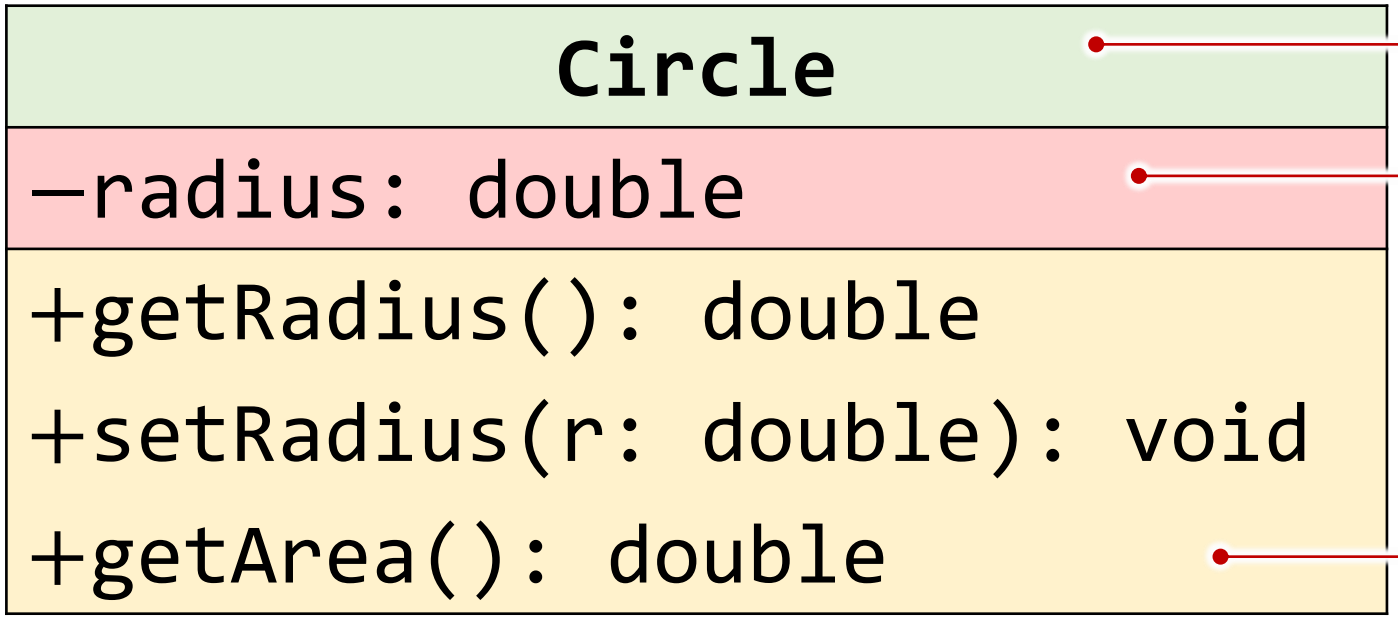
- Member Attributes / data fields / properties:
  - Member properties of a class



- Member Methods / behaviors / functions:
  - Member functions of a class



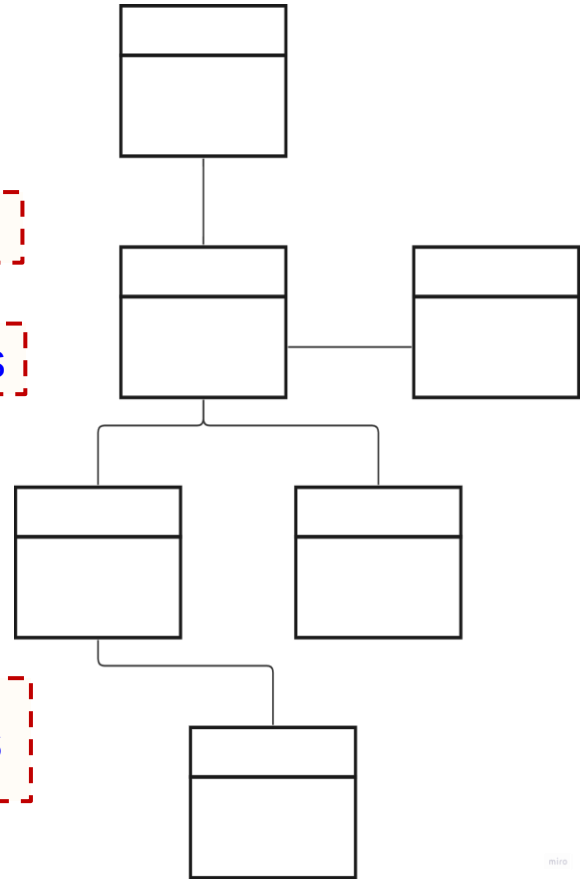
UML – Class Diagram



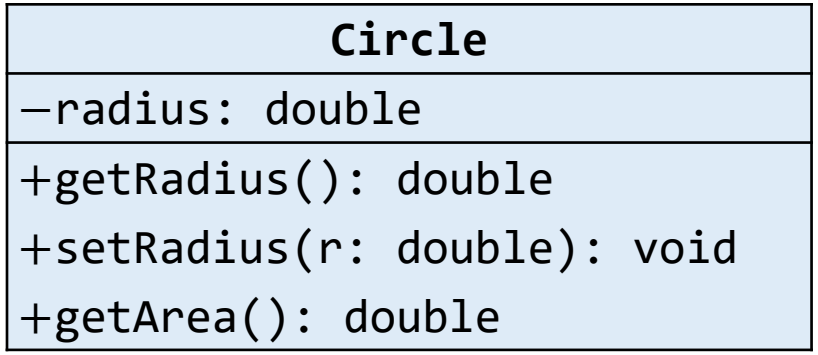
Class name

Member Attributes

Member Methods



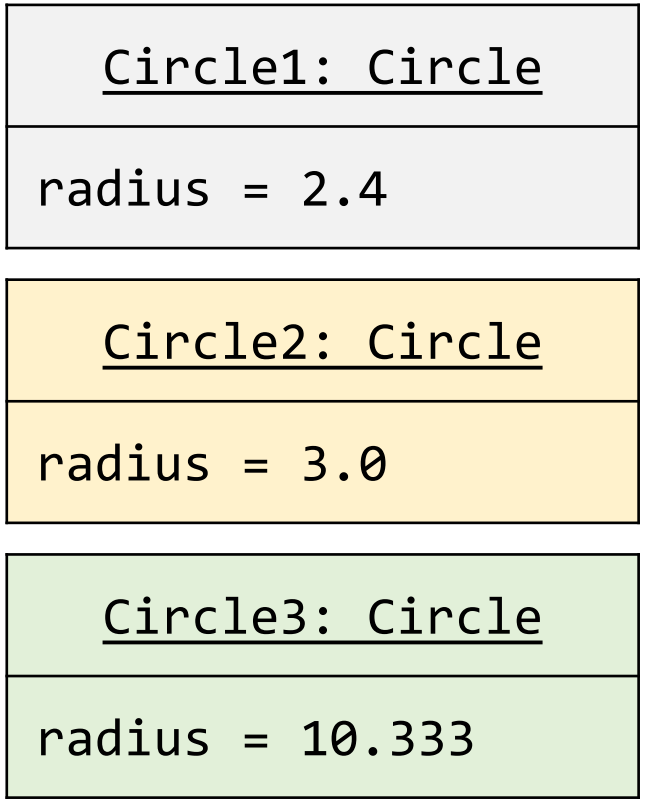
UML – Class Diagram



A class (a template / blueprint / …)

A construct that defines objects of the same type.

Three objects of the `Circle` class



Defines the object

State: A set of data fields (properties) with their current values.

Defines what the object does

Behavior: defined by a set of functions.

How to Declare a Class?

Class Example

```
class Circle
{
private:
    double radius = 1;

public:
    double getArea();
}; // Must place a semicolon here
```

Optional default value

Member data field

Member function

Circle
–radius: double
+getArea(): double

Default is private

Access Specifiers / Modifiers
private / public

Class Example

Like Global, but inside the object

```
class Circle
{
private:
   double radius = 1;

public:
   double getArea()
   {
      return radius * radius * 3.14159;
   }
}; // Must place a semicolon here
```

Can access member data field

Circle
–radius: double
+getArea(): double

```
int main()
{
   Circle c1;
   c1.radius = 5;
   cout << c1.getArea();
}
```

Class Example

Like Global, but inside the object

```
class Circle
{
public:
   double radius = 1;


   double getArea()
   {
      return radius * radius * 3.14159;
   }
}; // Must place a semicolon here
```

Circle
–radius: double
+getArea(): double

Can access member data field

```
int main()
{
   Circle c1;
   c1.radius = 5;
   cout << c1.getArea();
}
```

Class Example

Like Global, but inside the object

```
class Circle
{
public:
   double radius = 1;


   double getArea();

}; // Must place a semicolon here
```

Circle
−radius: double
+getArea(): double

Can access member data field

```
double Circle::getArea()
{
   return radius * radius * 3.14159;
}
```

```
int main()
{
   Circle c1;
   c1.radius = 5;
   cout << c1.getArea();
}
```

```
int main()
{
  Circle circle1;
  Circle circle2; circle2.radius = 25;
  Circle circle3; circle3.radius = 125;

  cout << "The area of the circle of radius "
    << circle1.radius << " is " << circle1.getArea() << endl;
  cout << "The area of the circle of radius "
    << circle2.radius << " is " << circle2.getArea() << endl;
  cout << "The area of the circle of radius "
    << circle3.radius << " is " << circle3.getArea() << endl;

  // Modify circle radius
  circle2.radius = 100;
  cout << "The area of the circle of radius "
    << circle2.radius << " is " << circle2.getArea() << endl;
}
```

Class name

Object name

```
class Circle
{
 public:
    double radius = 1;
    double getArea()
    {
        return radius * radius * 3.14159;
    }
}; // Must place a semicolon here
```

Object member access operator

Independent

The Scope of Data fields

Data fields and functions can be declared in any order in a class.

```
class Circle
{
private:
    double radius;

public:
    double getArea();
};
```

```
class Circle
{
public:
    double getArea();

private:
    double radius;
};
```

The Scope of Data fields

Data fields and functions can be declared in any order in a class.

```
class Circle
{
private:
    double radius;

public:
    double getArea();
    double memFun2();

public:
    void memFun3(double);
};
```

```
class Circle
{
public:
    double getArea();

private:
    double radius;

public:
    double memFun2();
    void memFun3(double);
};
```

```
class Circle
{
public:
    double getArea();
    double memFun2();
    void memFun3(double);

private:
    double radius;
};
```

The Scope of Data fields

Data fields are accessible to all member functions in the class (like global variables but inside the class).

Circle
−radius: double
+getArea(): double +setRadius(r: double): void +getRadius(): double

```
class Circle
{
public:
        double getArea()
        {
                return radius*radius*3.14;
        };
        double getRadius()
        {
                return radius;
        };
        void setRadius(double r)
        {
                radius = r;
        };

private:
        double radius;
};
```

Getter

Setter

Encapsulation

```
int main()
{
    Circle c1;
    cout << c1.getArea() << endl;
    c1.setRadius(10);
    cout << c1.getArea() << endl;
    cout << c1.getRadius() << endl;
}
```

The Scope of Local Variables

Local variables are declared and used inside a function locally.

Circle
−radius: double
+getArea(): double +setRadius(r: double): void +getRadius(): double

```
class Circle
{
public:
        double getArea()
        {
            double a;
            a = radius * radius * 3.14;
            return a;
        };
        double getRadius()
        {
                return radius;
        };
        void setRadius(double r)
        {
                radius = r;
        };

private:
        double radius;
};
```

The Scope of Local Variables

Circle
−radius: double
+getArea(): double +setRadius(r: double): void +getRadius(): double

Local variables are declared and used inside a function locally.

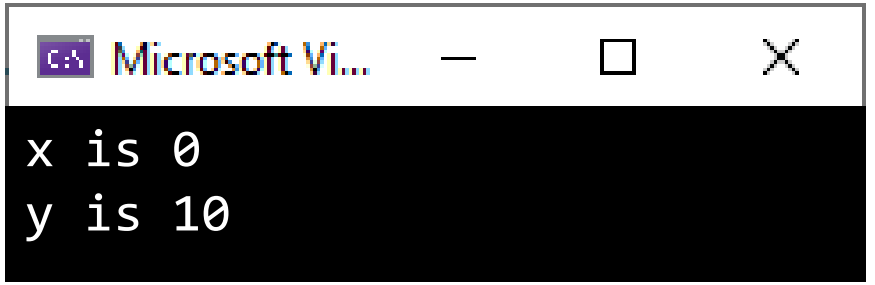
If a local variable has the same name as a data field, the local variable takes precedence and the data field with the same name is hidden.

```
class Circle
{
public:
        double getArea() { ... }
        double getRadius() { ... }
        void setRadius(double r)
        {
          double radius = r;
        };

private:
        double radius;
};
```

The Scope of Local Variables

```
int x; // Global variable

class Test
{
public:
  int y = 10; // Data fields

  void print()
  {

    cout << "x is " << x << endl;
    cout << "y is " << y << endl;
  }
};
```

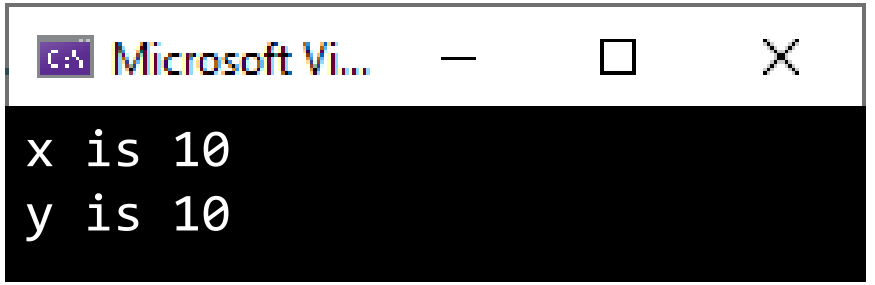
```
int main()
{
  Test test;
  test.print();

  return 0;
}
```



The Scope of Local Variables

```
int x; // Global variable

class Test
{
public:
  int x = 10, y = 10; // Data fields

  void print()
  {

    cout << "x is " << x << endl;
    cout << "y is " << y << endl;
  }
};
```

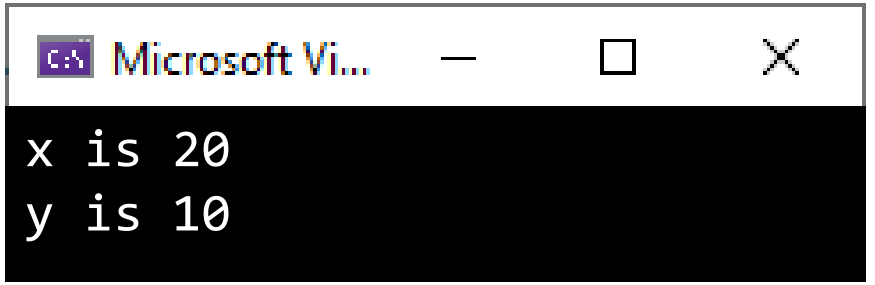
```
int main()
{
  Test test;
  test.print();

  return 0;
}
```



The Scope of Local Variables

```
int x; // Global variable

class Test
{
public:
  int x = 10, y = 10; // Data fields

  void print()
  {
    int x = 20; // Local variable
    cout << "x is " << x << endl;
    cout << "y is " << y << endl;
  }
};
```

```
int main()
{
  Test test;
  test.print();

  return 0;
}
```

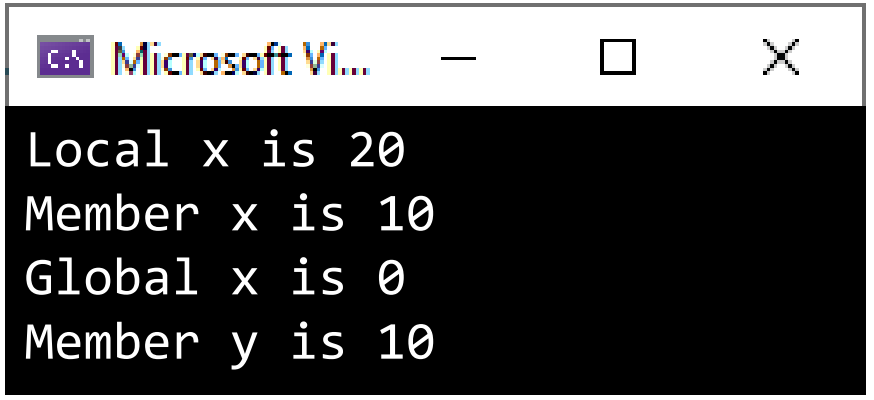


Can you access all three x's?

The Scope of Local Variables

```
int x; // Global variable

class Test
{
public:
  int x = 10, y = 10; // Data fields

  void print()
  {
    int x = 20; // Local variable
    cout << "Local x is " << x << endl;
    cout << "Member x is " << Test::x << endl;
    cout << "Global x is " << ::x << endl;
    cout << "Member y is " << y << endl;
  }
};
```

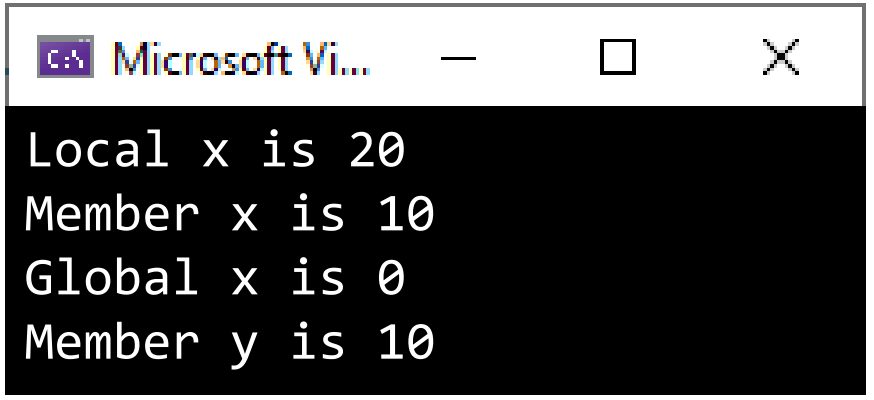
```
int main()
{
  Test test;
  test.print();

  return 0;
}
```



The Scope of Local Variables

```
int x; // Global variable

class Test
{
public:
  int x = 10, y = 10; // Data fields

  void print()
  {
    int x = 20; // Local variable
    cout << "Local x is " << x << endl;
    cout << "Member x is " << this->x << endl;
    cout << "Global x is " << ::x << endl;
    cout << "Member y is " << y << endl;
  }
};
```

```
int main()
{
  Test test;
  test.print();

  return 0;
}
```



Naming Objects and Classes

- When you declare a custom class, CAPITALIZE the first letter of each word in a class name; for example, the class names `Circle`, `Rectangle`, and `GeometricShape`.
- The class names in the C++ library are named in lowercase: `string`, `iostream`
- The objects are named like variables.

```
TV tv1;
Car myCar;
```

Memberwise Copy

When use the assignment operator `=` , each data field of one object is copied to its counterpart in the other object.

Circle
−radius: double
+getArea(): double ...

```
int main()
{
   Circle c1, c2;
   c2 = c1; // Copies radius in c1 to c2
}
```

After the copy, `c1` and `c2` are still two different objects, but with the same radius value.

However …

```
bool b = (c1 == c2); // Will not work by default
```

Abstraction and Encapsulation

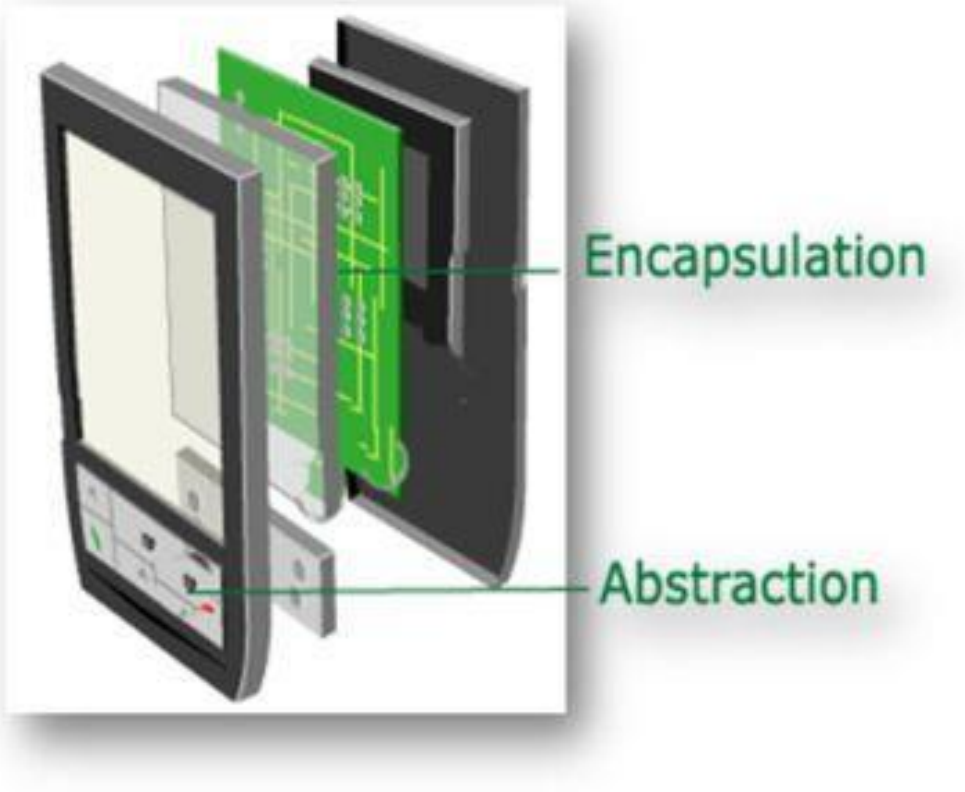
Class Abstraction and Encapsulation

Class **Abstraction**

- to **separate** class implementation from the use of the class.
- only the **essential** details are displayed to the user
- Define the high-level features and functionalities without the details of how.

Class **Encapsulation**

- to bundle the data and methods related to an object into a single unit.
- to allow using a protective shield to **prevent** accessing the data by the code outside this shield.



Concepts of Encapsulation and Abstraction (C#) (completecsharptutorial.com)

Data Field Encapsulation

If the data fields are public, they can be modified directly

```
circle1.radius = -5
student5.gpa = 9.5
client.accountBalance += 5e512
```

Not a good practice:

- Data may be tampered.
- Makes the class difficult to maintain and vulnerable to bugs.

Use Getter / Setter

Example – Getter (Accessor) and Setter (Mutator)

```
class TV
{
private:
  int channel = 1, volumeLevel = 1;
  bool on = false;

public:
  // Getters (Accessors):
  // returnType getPropertyName()
  int getChannel();
  bool isOn();
  …

  // Setters (Mutators):
  // void setPropertyName(dataType value)
  void setChannel(int newChannel);
  void TurnOn();
  void TurnOff();
  …
};
```

```
int TV::getChannel()
{
     return channel;
}

void TV::setChannel(int newChannel)
{
     channel = newChannel;
}

bool TV::isOn()
{
     return on;
}

void TV::TurnOn()
{
     on = true;
}

void TV::TurnOff()
{
     on = false;
}
```

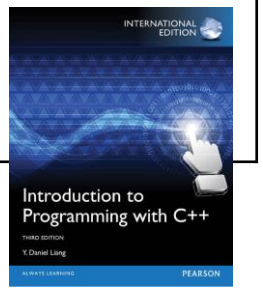
Is that it?

Example – Getter (Accessor) and Setter (Mutator)

Private

TV	
−channel: int	The current channel (1 to 120) of this TV.
−volumeLevel: int	The current volume level (1 to 7) of this TV.
−on: Boolean	Indicates whether this TV is on/off.
+turnOn(): void	Turns on this TV.
+turnOff(): void	Turns off this TV.
+setChannel(newChannel: int): void	Sets a new channel for this TV.
+setVolume(newVolume: int): void	Sets a new volume level for this TV.
+channelUp(): void	Increases the channel number by 1.
+channelDown(): void	Decreases the channel number by 1.
+volumeUp(): void	Increases the volume level by 1.
+volumeDown(): void	Decreases the volume level by 1.

Public



Example – Getter (Accessor) and Setter (Mutator)

```
class TV
{
private:
   int channel = 1, volumeLevel = 1;
   bool on = false;

public:
   void TurnOn(); void TurnOff();
   void setChannel(int newChannel);
   void setVolume (int newVolume);
   void channelUp(); void channelDown();
   void volumeUp();  void volumeDown();
};
```

TV
−channel: int −volumeLevel: int −on: Boolean
+turnOn(): void +turnOff(): void +setChannel(newChannel: int): void +setVolume(newVolume: int): void +channelUp(): void +channelDown(): void +volumeUp(): void +volumeDown(): void

The current channel (1 to 120) of this TV. The current volume level (1 to 7) of this TV. Indicates whether this TV is on/off.
Turns on this TV. Turns off this TV. Sets a new channel for this TV. Sets a new volume level for this TV. Increases the channel number by 1. Decreases the channel number by 1. Increases the volume level by 1. Decreases the volume level by 1.

Example – Getter (Accessor) and Setter (Mutator)

```
void TV::turnOn()
{
   on = true;
}

void TV::turnOff()
{
   on = false;
}
```

```
class TV
{
private:
  int channel = 1, volumeLevel = 1;
  bool on = false;

public:
  void TurnOn(); void TurnOff();
  void setChannel(int newChannel);
  void setVolume (int newVolume);
  void channelUp(); void channelDown();
  void volumeUp();  void volumeDown();
};
```

The current channel (1 to 120) of this TV. The current volume level (1 to 7) of this TV. Indicates whether this TV is on/off.
Turns on this TV. Turns off this TV. Sets a new channel for this TV. Sets a new volume level for this TV. Increases the channel number by 1. Decreases the channel number by 1. Increases the volume level by 1. Decreases the volume level by 1.

Example – Getter (Accessor) and Setter (Mutator)

```
void TV::setChannel(int newChannel)
{
  if (on && newChannel >= 1
         && newChannel <= 120)
    channel = newChannel;
}

void TV::setVolume(int newVolumeLevel)
{
  if (on && newVolumeLevel >= 1
         && newVolumeLevel <= 7)
    volumeLevel = newVolumeLevel;
}
```

```
class TV
{
private:
  int channel = 1, volumeLevel = 1;
  bool on = false;

public:
  void TurnOn(); void TurnOff();
  void setChannel(int newChannel);
  void setVolume (int newVolume);
  void channelUp(); void channelDown();
  void volumeUp();  void volumeDown();
};
```

The current channel (1 to 120) of this TV. The current volume level (1 to 7) of this TV. Indicates whether this TV is on/off.
Turns on this TV. Turns off this TV. Sets a new channel for this TV. Sets a new volume level for this TV. Increases the channel number by 1. Decreases the channel number by 1. Increases the volume level by 1. Decreases the volume level by 1.

Example – Getter (Accessor) and Setter (Mutator)

```
void TV::channelUp()
{
  if (on && channel < 120)
    channel++;
}

void TV::channelDown()
{
  if (on && channel > 1)
    channel--;
}
```

```
class TV
{
private:
  int channel = 1, volumeLevel = 1;
  bool on = false;

public:
  void TurnOn(); void TurnOff();
  void setChannel(int newChannel);
  void setVolume (int newVolume);
  void channelUp(); void channelDown();
  void volumeUp();  void volumeDown();
};
```

The current channel (1 to 120) of this TV.
The current volume level (1 to 7) of this TV.
Indicates whether this TV is on/off.

Turns on this TV.
Turns off this TV.
Sets a new channel for this TV.
Sets a new volume level for this TV.
Increases the channel number by 1.
Decreases the channel number by 1.
Increases the volume level by 1.
Decreases the volume level by 1.

Example – Getter (Accessor) and Setter (Mutator)

```
void TV::volumeUp()
{
  if (on && volumeLevel < 7)
    volumeLevel++;
}

void TV::volumeDown()
{
  if (on && volumeLevel > 1)
    volumeLevel--;
}
```

```
class TV
{
private:
  int channel = 1, volumeLevel = 1;
  bool on = false;

public:
  void TurnOn(); void TurnOff();
  void setChannel(int newChannel);
  void setVolume (int newVolume);
  void channelUp(); void channelDown();
  void volumeUp();  void volumeDown();
};
```

The current channel (1 to 120) of this TV.
The current volume level (1 to 7) of this TV.
Indicates whether this TV is on/off.

Turns on this TV.
Turns off this TV.
Sets a new channel for this TV.
Sets a new volume level for this TV.
Increases the channel number by 1.
Decreases the channel number by 1.
Increases the volume level by 1.
Decreases the volume level by 1.

Example – Getter (Accessor) and Setter (Mutator)

```
int main()
{
   TV tv1;
   tv1.turnOn();
   tv1.setChannel(30);
   tv1.setVolume(6);

   TV tv2; tv2.turnOn();
   tv2.channelUp(); tv2.channelUp();
   tv2.volumeUp(); tv2.volumeUp(); tv2.volumeUp();

   return 0;
}
```

```
class TV
{
private:
  int channel = 1, volumeLevel = 1;
  bool on = false;

public:
  void TurnOn(); void TurnOff();
  void setChannel(int newChannel);
  void setVolume (int newVolume);
  void channelUp(); void channelDown();
  void volumeUp();  void volumeDown();
};
```

Constant and Static Members

Constant Members

You can use `const` to specify a constant parameter.

```
class Circle
{
public:
  double getArea() const;
  double getRadius() const;
  void setRadius(double);

private:
  double radius;
  const double MAX_RADIUS = 5;
};
```

```
int main()
{
    Circle c1;
    c1.setRadius(10);
    cout << c1.getArea();
}
```

Constant Members

You can use `const` to specify a constant parameter.

You also can specify a constant member function

- The function **should not** change the value of any member data fields in the object

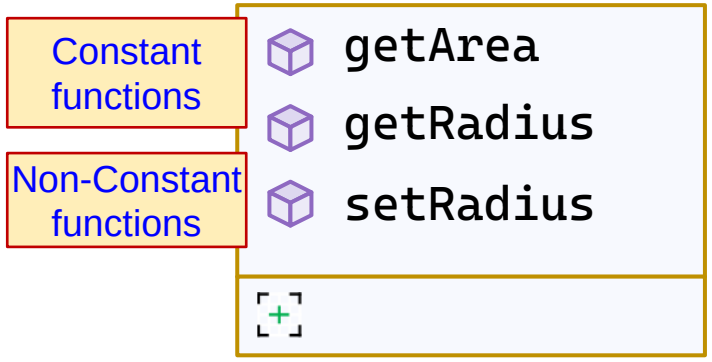
```
class Circle
{
public:
  double getArea() const;
  double getRadius() const;
  void setRadius(double);

private:
  double radius;
  const double MAX_RADIUS = 5;
};
```

Typically, Getters are `const`

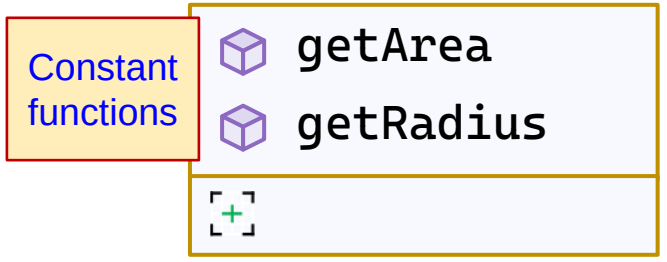
```
int main()
{
    Circle c1;
    c1.setRadius(10);
    cout << c1.getArea();
}
```

Constant Members

```
int main()
{
    Circle regularCircle;
    const Circle constCircle;

    regularCircle.|

    return 0;
}
```




```
int main()
{
    Circle regularCircle;
    const Circle constCircle;

    regularCircle.setRadius(3.5);
    constCircle.|

    return 0;
}
```



Constant Members

```
double Circle::getArea()
{
   return radius * radius * 3.14;
}
```

```
class Circle
{
public:
   double getArea();
   double getRadius();
   void setRadius(double);
private:
   double radius;
};
```

This code **will not compile** if the `getArea()` function is **not** a `const` function.

```
void printCircleArea(const Circle& c) // Global function
{
    cout << "The area of the circle is " << c.getArea();
}
```

Constant Members

```
double Circle::getArea() const
{
   return radius * radius * 3.14;
}
```

```
class Circle
{
public:
   double getArea() const;
   double getRadius() const;
   void setRadius(double);
private:
   double radius;
};
```

This code **will compile** if the `getArea()` function is a `const` function.

```
void printCircleArea(const Circle& c) // Global function
{
    cout << "The area of the circle is " << c.getArea();
}
```

If you have a `const` object
- you can only use its `const` member functions
- You can not use its non-`const` member functions

Static Member Variables

You can have Static Member Variables

- All objects of the class share the same instance of this variable.
- All get affected when changes
- Must define outside the class
- Initialize only outside the class
- Defaults to zero (or chosen value) with the first created object

```
class Grade
{
public:
    int static minGrade;
    int grade = 10;
};

int Grade::minGrade = 60;

int main()
{
    cout << Grade::minGrade;

    Grade g;
    cout << g.grade;
}
```

Declaration

Definition

Initialization

Static Member Functions

- Can only access static member variables and static functions
- As any function, can have its own local variables
- Can be called without creating an instance of the class.

```
int main()
{
    Grade::setMinGrade(20);
}
```

```
class Grade
{
private:
    int static minGrade;
    int grade;

public:
    void setGrade(int v);
    double getGrade() const;
    static void setMinGrade(int v);
    static int getMinGrade();
};

int Grade::minGrade = 60;
```

Static Members

Grade.cpp

```cpp
#include "Grade.h"

// Can access static & non-static members
void Grade::setGrade(int v) { grade = v; }
double Grade::getGrade() const { return grade; }
bool Grade::isPassing() const
{
    return grade >= minGrade;
}
string Grade::getPF() const
{
    return ( isPassing() ? "P" : "F" );

}

// Can ONLY access static members
void Grade::setMinGrade(int v) { minGrade = v; }
int Grade::getMinGrade() { return minGrade; }
```

Grade.h

```cpp
#pragma once ?

class Grade
{
private:
    int static minGrade;
    int grade;

public:
    void setGrade(int v);
    double getGrade() const;
    bool isPassing() const;
    string getPF() const;
    static void setMinGrade(int v);
    static int getMinGrade();
};

int Grade::minGrade = 60;
```

Static Members

```
int main()
{
    Grade s[10];
    for (int i = 0; i < 10; i++)
        s[i].setGrade(10 * i + 1);

    for (int i = 0; i < 10; i++)
        cout << s[i].getGrade() << ": "
             << s[i].getPF() << endl;

    cout << string(4, '-') << endl;

    Grade::setMinGrade(20);
    for (int i = 0; i < 10; i++)
        cout << s[i].getGrade() << ": "
             << s[i].getPF() << endl;

    return 0;
}
```

Grade.h

```
#pragma once

class Grade
{
private:
    int static minGrade;
    int grade;

public:
    void setGrade(int v);
    double getGrade() const;
    bool isPassing() const;
    string getPF() const;
    static void setMinGrade(int v);
    static int getMinGrade();
};

int Grade::minGrade = 60;
```

Static Membe

```
int main()
{
    Grade s[10];
    for (int i = 0; i < 10; i++)
        s[i].setGrade(10 * i + 1);

    for (int i = 0; i < 10; i++)
        cout << s[i].getGrade() << ": "
             << s[i].getPF() << endl;

    cout << string(4, '-') << endl;
    s[3].setMinGrade(20);
    Grade::setMinGrade(20);
    for (int i = 0; i < 10; i++)
        cout << s[i].getGrade() << ": "
             << s[i].getPF() << endl;

    return 0;
}
```

Better

```
#p

cl
{
pr

pu

};

in
```

Microsoft Vi...

```
1: F
11: F
21: F
31: F
41: F
51: F
61: P
71: P
81: P
91: P
----
1: F
11: F
21: P
31: P
41: P
51: P
61: P
71: P
81: P
91: P
```

Instance or Static Member Function?

How to decide?

- A variable or function that is dependent on a specific instance of the class should be an **instance** variable or function.
- A variable or function that is not dependent on a specific instance of the class should be a **static** variable or function.

```
class Grade
{
private:
    int static minGrade;
    int grade;

public:
    void setGrade(int v);
    double getGrade() const;
    bool isPassing() const;
    string getPF() const;
    static void setMinGrade(int v);
    static int getMinGrade();
};

int Grade::minGrade = 60;
```

Static member

Instance member

Instance member

Static member

Instance or Static Member Function?

How to decide?

- A variable or function that is dependent on a specific instance of the class should be an **instance** variable or function.
- A variable or function that is not dependent on a specific instance of the class should be a **static** variable or function.

```
class Grade
{
private:
    int static minGrade;
    int grade;

public:
    void setGrade(int v);
    double getGrade() const;
    bool isPassing() const;
    string getPF() const;
    static void setMinGrade(int v);
    static int getMinGrade();
};

int Grade::minGrade = 60;
```

Can have `static const` member variable

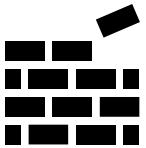
Can not have `static const` member function

Static Members

```
class AppConfig
{
public:
     static const int    verMajor;
     static const int    verMinor;
     static const string themeColor;
     static const int    MAX_USERS;

     // Static function to display application configuration
     static void displayConfig()
     {
          cout << "Application Version: "     << verMajor   << "." << verMinor << endl;
          cout << "Theme Color: "             << themeColor << endl;
          cout << "Maximum Number of Users: " << MAX_USERS  << endl;
     }
};

// Initialize static members
const int    AppConfig::verMajor = 1;
const int    AppConfig::verMinor = 0;
const string AppConfig::themeColor = "DarkBlue";
const int    AppConfig::MAX_USERS = 100;
```

```
int main()
{
    // Accessing without
    // instantiation
    AppConfig::displayConfig();
}
```

Constructors and Destructors

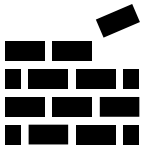


Constructors

- A special type of functions that are automatically called when an object is created.
  - Normally is used to initialize the object.
- Has the same name as the defining class.
- Does not have a return type—not even `void`.

```
class Circle
{
public:
1   double radius;

    Circle()
2   {
      radius = 1;
    }




};
```

```
int main()
{
    Circle c1;

}
```



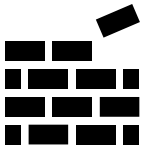
Constructors

- A special type of functions that are automatically called when an object is created.
  - Normally is used to initialize the object.
- Has the same name as the defining class.
- Does not have a return type—not even `void`.
- Can be overloaded to construct objects with different options.
- If you declare a class without constructors:
  - A **default constructor** with an empty body is implicitly declared.

```
class Circle
{
public:
1   double radius;

    Circle()
2   {
      radius = 1;
    }

    Circle(double newRadius)
2   {
      radius = newRadius;
    }
};
```

Overload

```
int main()
{
    Circle c1;
    Circle c2(10.5);
}
```

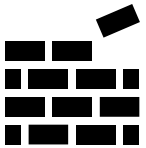


Constructors

- A special type of functions that are automatically called when an object is created.
  - Normally is used to initialize the object.
- Has the same name as the defining class.
- Does not have a return type—not even `void`.
- Can be overloaded to construct objects with different options.
- If you declare a class without constructors:
  - A **default constructor** with an empty body is implicitly declared.

```
class Circle
{
public:
   double radius;

   Circle()
   {
      radius = 1;
   }

   Circle(double newRadius)
   {
      radius = newRadius;
   }
};
```

```
int main()
{
    Circle c1;
    Circle c2(10.5);
}
```

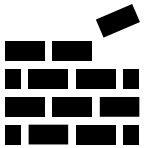


Constructors

- A special type of functions that are automatically called when an object is created.
  - Normally is used to initialize the object.
- Has the same name as the defining class.
- Does not have a return type—not even `void`.
- Can be overloaded to construct objects with different options.
- If you declare a class without constructors:
  - A **default constructor** with an empty body is implicitly declared.

```
class Circle
{
public:
   double radius;

   Circle()
   {
      radius = 1;
   }

   Circle(double newRadius)
   {
      radius = newRadius;
   }
};
```

```
int main()
{
    Circle c1;
    Circle c2(10.5);
}
```

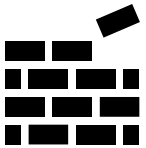


Constructors

- A special type of functions that are automatically called when an object is created.
  - Normally is used to initialize the object.
- Has the same name as the defining class.
- Does not have a return type—not even `void`.
- Can be overloaded to construct objects with different options.
- If you declare a class without constructors:
  - A **default constructor** with an empty body is implicitly declared.

```
class Circle
{
public:
   double radius;

   Circle()
   {
     radius = 1;
   }

   Circle(double newRadius)
   {
     radius = newRadius;
   }
};
```

```
int main()
{
    Circle c1;
    Circle c2(10.5);
}
```



Constructors

- A special type of functions that are automatically called when an object is created.
  - Normally is used to initialize the object.
- Has the same name as the defining class.
- Does not have a return type—not even `void`.
- Can be overloaded to construct objects with different options.
- If you declare a class without constructors:
  - A **default constructor** with an empty body is implicitly declared.
- You typically provide a no-arg / default constructor, and different parametrized constructor overloads.

```
class Circle
{
public:
   double radius;

   Circle()
   {
      radius = 1;
   }

   Circle(double newRadius)
   {
      radius = newRadius;
   }
};
```

```
int main()
{
    Circle c1;
    Circle c2(10.5);
}
```

What is wrong with this code?

```
class Circle
{
private:
        double radius;
public:
        Circle(double r) {...}
        void setRadius(double) {...}
        double getRadius() const {...}
        double getArea() const {...}
};

int main()
{
        Circle c1;
        c1.setRadius(5);
        cout << c1.getArea();
}
```

no default constructor exists for class Circle

~~What is wrong with this code?~~

```
class Circle
{
private:
        double radius;
public:
        Circle(double r = 1.0) {...}
        void setRadius(double) {...}
        double getRadius() const {...}
        double getArea() const {...}
};

int main()
{
        Circle c1;
        c1.setRadius(5);
        cout << c1.getArea();
}
```

Constructors & Static Members

Circle.h

```
#pragma once

class Circle
{
public:
    Circle(double = 1);
    double getArea() const;
    double getRadius() const;
    void setRadius(double);
    static int getNumOfObjects();

private:
    double radius;
    static int numOfObjects;
};
```

Circle.cpp

```
#include "Circle.h"

int Circle::numOfObjects = 0;

// Can access static members
// Can access non-static members
Circle::Circle(double newRadius)
{
    radius = newRadius;
    numOfObjects++;
}

// Can ONLY access static members
int Circle::getNumOfObjects()
{
    return numOfObjects;
}
...
```

Constructors & Static Members

```cpp
#include <iostream>
#include "Circle.h"
using namespace std;
int main()
{
    cout << "Num of circles created: "
         << Circle::getNumOfObjects() << endl;

    Circle circ1;
    cout << "Num of circles created: "
         << Circle::getNumOfObjects() << endl;

    Circle circ2(5.0);
    cout << "Num of circles created: "
         << circ1.getNumOfObjects() << endl;
}
```

Circle.cpp

```cpp
#include "Circle.h"

int Circle::numOfObjects = 0;

// Can access static members
// Can access non-static members
Circle::Circle(double newRadius)
{
    radius = newRadius;
    numOfObjects++;
}

// Can ONLY access static members
int Circle::getNumOfObjects()
{
    return numOfObjects;
}
...
```

Constructors & Static Members

```cpp
#include <iostream>
#include "Circle.h"
using namespace std;
int main()
{
    cout << "Num of circles created: "
         << Circle::getNumOfObjects() << endl;

    Circle circ1;
    cout << "Num of circles created: "
         << Circle::getNumOfObjects() << endl;

    Circle circ2(5.0);
    cout << "Num of circles created: "
         << circ1.getNumOfObjects() << endl;
}
```

Better

Circle.cpp

```cpp
#include "Circle.h"

int Circle::numOfObjects = 0;

// Can access static members
// Can access non-static members
Circle::Circle(double newRadius)
{
    radius = newRadius;
    numOfObjects++;
}
...
```

Microsoft Vi...

```
Num of circles created: 0
Num of circles created: 1
Num of circles created: 2
```

Destructors

- A member function that is automatically called when an object is destroyed
- Destructor name is `~classname`, *e.g.*, `~Rectangle`
- Has no return type; takes no arguments
- Only one destructor per class (no overloading)

```
class Rectangle
{
public:
   Rectangle(); // Constructor
   ~Rectangle(); // Destructor
   ...
};
```

Destructors

Circle.h

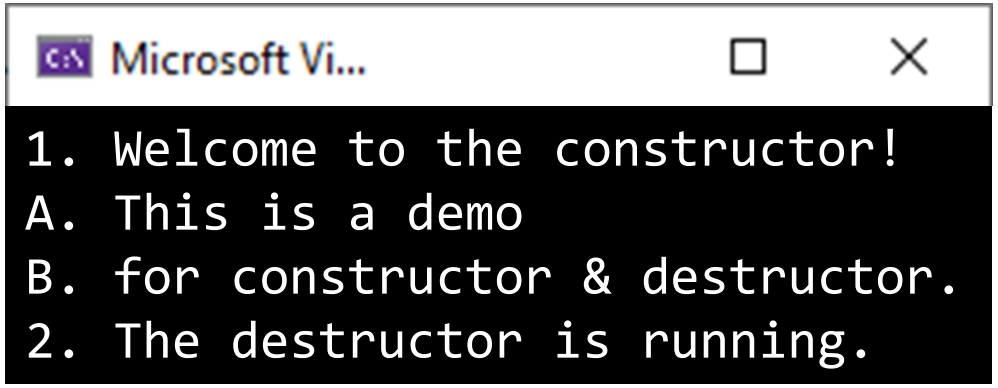
```
#pragma once

class Circle
{
public:
    Circle(double = 1);
    ~Circle();

    double getArea() const;
    double getRadius() const;
    void setRadius(double);
    static int getNumOfObjects();

private:
    double radius;
    static int numOfObjects;
};
```

Circle.cpp

```
#include "Circle.h"

int Circle::numOfObjects = 0;

Circle::Circle(double newRadius)
{
    radius = newRadius;
    numOfObjects++;
}

int Circle::getNumOfObjects()
{
    return numOfObjects;
}

Circle::~Circle()
{
    numOfObjects--;
}
```

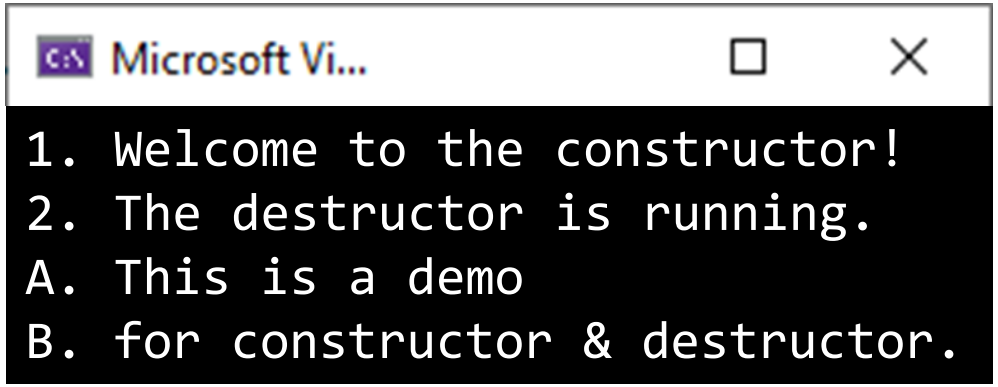
Destructors

```
class Demo
{
public:
    Demo() // Constructor
    {
        cout << "1. Welcome to the constructor!\n";
    }
    ~Demo() // Destructor
    {
        cout << "2. The destructor is running.\n";
    }
};

int main()
{
    Demo demoObject;

    cout << "A. This is a demo\n";
    cout << "B. for constructor & destructor.\n";
}
```



Destructors

```
class Demo
{
public:
    Demo() // Constructor
    {
        cout << "1. Welcome to the constructor!\n";
    }
    ~Demo() // Destructor
    {
        cout << "2. The destructor is running.\n";
    }
};

void SomeGlobalFunction() { Demo demoObject; }

int main()
{
    SomeGlobalFunction();

    cout << "A. This is a demo\n";
    cout << "B. for constructor & destructor.\n";
}
```



Anonymous Object

You may create an object and use it only once without the need to name it (*anonymous object*)

```
cout << "The Area for a circle of radius 5 is "
  << Circle(5).getArea() << endl;
```

with-arg

```
cout << "The Area for a circle of the default radius is "
  << Circle().getArea() << endl;
```

no-arg

Initializing Member Data Fields

Initializing Member Data Fields

```
Rectangle::Rectangle()
{
}
```

Default member initializers

```
class Rectangle
{
private:
   double length = 1;
   double width = 1;
   …
```

```
int main()
{
    Rectangle r1;
}
```

Initializing Member Data Fields

```
class Rectangle
{
private:
   double length;
   double width;
   …
```

```
Rectangle::Rectangle(double l, double w)
{
      length = l;
      width = w;
}
```

Constructor initialization

```
int main()
{
    Rectangle r1;
}
```

```
Rectangle::Rectangle()
{
      length = 1;
      width = 1;
}
```

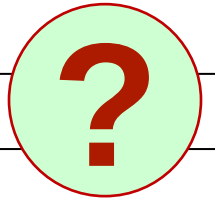
Initializing Member Data Fields

```
Rectangle::Rectangle(double l, double w)
{
      length = l;
      width = w;
}
```

Constructor initialization

```
Rectangle::Rectangle()
{
      length = 1;
      width = 1;
}
```

```
class Rectangle
{
private:
   double length = 0;
   double width = 0;
   …
```




```
int main()
{
    Rectangle r1;
}
```

Initializing Member Data Fields

```
class Rectangle
{
private:
   double length, width;
   …
```

```
Rectangle::Rectangle(double l, double w) :
      length(l), width(w)
{

}
```

Initialization lists

```
int main()
{
    Rectangle r1;
}
```

```
Rectangle::Rectangle() :
      length(1), width(1)
{

}
```

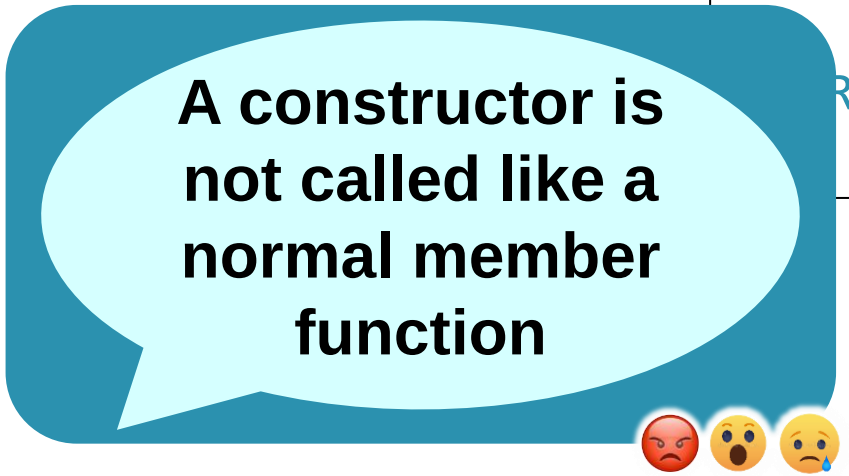
Initializer list is where you can initialize constant and reference member variables

Initializing Member Data Fields

```
class Rectangle
{
private:
   double length, width;
   …
```

```
Rectangle::Rectangle(double l, double w) :
      length(l), width(w)
{

}
```

```
int main()

   Rectangle r1;
```




```
Rectangle::Rectangle()
{
    Rectangle(1, 1); // Don't do that!!
      // This creates an extra unused object
}
```

Initializing Member Data Fields

```
class Rectangle
{
private:
   double length, width;
   …
```

```
Rectangle::Rectangle(double l, double w) :
      length(l), width(w)
{

}
```

```
int main()
{
    Rectangle r1;
    Rectangle r2(2,3);

}
```

```
Rectangle::Rectangle() : Rectangle(1, 1)
{


}
```

Constructor Delegation

Initializing Member Data Fields

```
class Rectangle
{
private:
   double length, width;
   …
```

```
Rectangle::Rectangle(double l, double w) :
      length(l), width(w)
{
   // Any other main steps
}

Rectangle::Rectangle() : Rectangle(1, 1)
{
}

Rectangle::Rectangle(double l) : Rectangle(l, l)
{
}
```

```
int main()
{
    Rectangle r1;
    Rectangle r2(2,3);
    Rectangle r3(2);
}
```

Constructor Delegation

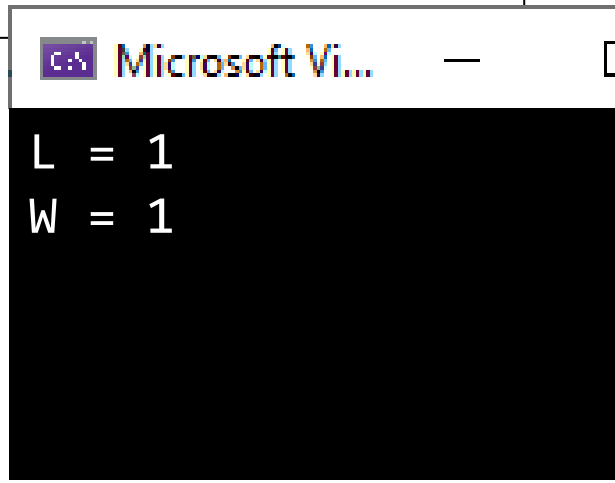
Initializing Member Data Fields

```
class Rectangle
{
private:
   double length, width;
   …
```

```
Rectangle::Rectangle(double l, double w)
{
      setLength(l);
      setWidth(w);
}
```

```
int main()
{
    Rectangle r1;
    cout << "L = " << r1.getLength() << "\n";
    cout << "W = " << r1.getWidth() << "\n";
}
```

```
Rectangle::Rectangle() : Rectangle(l, l)
{


}
```



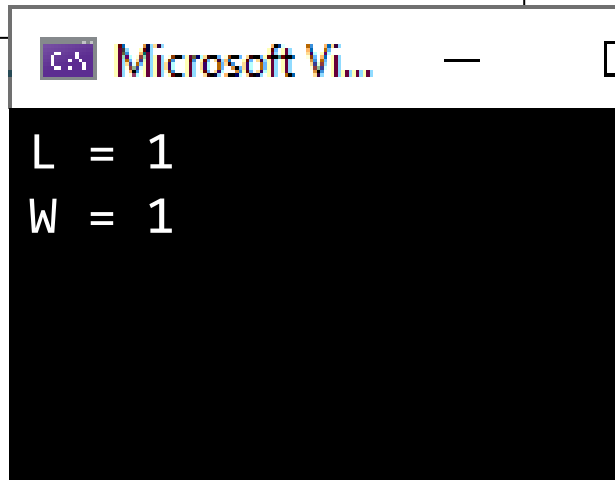
READ

Initializing Member Data Fields

```
class Rectangle
{
private:
   double length, width;
   …
```

```
Rectangle::Rectangle(double l, double w)
{
      initRectangle(l, w);
}
Rectangle::Rectangle()
{
      initRectangle(1, 1);
}
void Rectangle::initRectangle(double l, double w)
{
      setLength(l);
      setWidth(w);
}
```

```
int main()
{
   Rectangle r1;
   cout << "L = " << r1.getLength() << "\n";
   cout << "W = " << r1.getWidth() << "\n";
}
```



READ

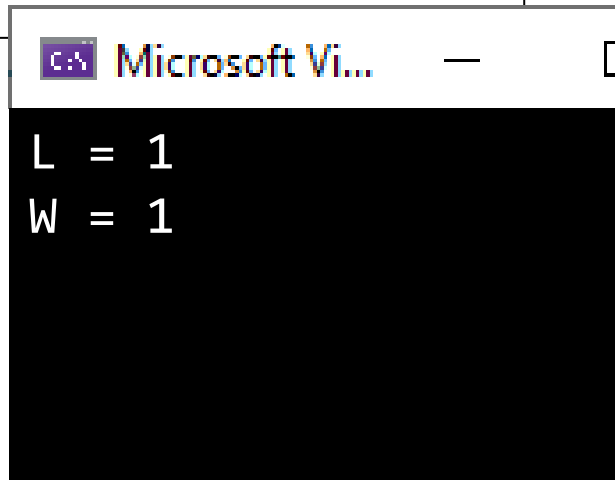
Initializing Member Data Fields

```
class Rectangle
{
private:
   double length, width;
   …
```

```
Rectangle::Rectangle(double l = 1, double w = 1)
{
      setLength(l);
      setWidth(w);
}
```

```
int main()
{
   Rectangle r1;
   cout << "L = " << r1.getLength() << "\n";
   cout << "W = " << r1.getWidth() << "\n";
}
```

Use default parameters



READ

Initializing Member Data Fields

How to initialize reference variables?

```
class Rectangle
{
private:
    double width = 2;
    double length = 1;
public:
    const double& Width = width;
    const double& Length = length;
    Rectangle() {};
};
```

```
int main()
{
    Rectangle r;
    cout << r.Length;
    r.Length = 9;
}
```

Read-only version of hidden length

READ

Initializing Member Data Fields

How to initialize reference variables?

```
class Rectangle
{
private:
    double width = 2;
    double length = 1;
public:
    const double& Width;
    const double& Length;
    Rectangle() : Width(width), Length (length) {};
};
```

```
int main()
{
    Rectangle r;
    cout << r.Length;
    r.Length = 9;
}
```

Read-only version of hidden length

READ

DO IT YOURSELF!!

Initializing Member Data Fields

```
class Rectangle
{
private:
   double width  = 2;
   double length = 1;
```

What is the output of each of these alternative?

```
Rectangle::Rectangle() : length(10), width(20) { }

Rectangle::Rectangle() : length(10), width(length) { }

Rectangle::Rectangle() : length(width), width(20) { }
```

```
int main()
{
    Rectangle r1;
    cout << "L = " << r1.getLength() << "\n";
    cout << "W = " << r1.getWidth() << "\n";
}
```

Objects with Functions and Arrays

Passing Objects to Functions

You can pass objects by value or by reference, but it is more efficient to pass objects by reference or constant reference

```
void displayCircle(const Circle& c)
{
     cout << "Radius: " << c.getRadius() << endl;
     cout << "Area: " << c.getArea() << endl;
}

void initializeCircle(Circle& c, double r)
{
     c.setRadius(r);
}

int main()
{
     Circle c(5);
     initializeCircle(c, 10);
     displayCircle(c);
}
```

```
class Circle
{
public:
     Circle(double r = 1)
       : radius(r) {}

     double getArea() const;
     double getRadius() const;
     void setRadius(double r);

private:
     double radius;
};
```

Passing Objects to Functions

```
class BankAccount
{
private:
     double balance;
public:
     BankAccount(double initBal) : balance(initBal) {}

     bool hasSameBalance(const BankAccount& other) const
     {
          return balance == other.balance;
     }
};
```

```
int main()
{
     BankAccount account1(5000.00), account2(3000.00);
     if (account1.hasSameBalance(account2))
          cout << "Same balance." << endl;
     else
          cout << "Different balances." << endl;

}
```

Passing Objects to Functions

```
class BankAccount
{
private:
     double balance;
public:
     BankAccount(double initBal) : balance(initBal) {}

     bool hasSameBalance(const BankAccount& other) const
     {
          return this->balance == other.balance;
     }
};
```

```
int main()
{
     BankAccount account1(5000.00), account2(3000.00);
     if (account1.hasSameBalance(account2))
          cout << "Same balance." << endl;
     else
          cout << "Different balances." << endl;

}
```

Array of Objects

```
int main()
{
  const int SIZE = 10;

  Circle circleArray[SIZE];

  for (int i = 0; i < SIZE; i++)
  {
    circleArray[i].setRadius(i + 1);
  }

  cout << "The total area of circles is "
       << sum(circleArray, SIZE) << endl;

  return 0;
}
```

Default constructor is called per item

```
class Circle
{
public:
  Circle(double newRadius = 1)
     : radius(newRadius) {}
  double getArea() const;
  double getRadius() const;
  void setRadius(double);

private: double radius;
};
```

```
double sum(const Circle cArray[], int N)
{
  double sum = 0;
  for (int i = 0; i < N; i++)
    sum += cArray[i].getArea();
  return sum;
}
```

Array of Objects

```
class Inventory
{
public:
     Inventory(string newName, double newSize) : name(newName), size(newSize)
     {
          cout << "You are using: Inventory(string, double)" << endl;
     }
     Inventory(string newName) : Inventory(newName, 0.0)
     {
          cout << "You are using: Inventory(string)" << endl;
     }
     Inventory(double newSize) : Inventory("Tool", newSize)
     {
          cout << "You are using: Inventory(double)" << endl;
     }
     Inventory() : Inventory("Tool", 0.0)
     {
          cout << "You are using: Inventory()" << endl;
     }
private:
     string name;
     double size;
};
```

```
int main()
{
     Inventory myInventoryAr1[5];
}
```

Array of Objects

```
class Inventory
{
public:
     Inventory(string newName, double newSize) : name(newName), size(newSize)
     {
          cout << "You are using: Inventory(string, double)" << endl;
     }
     Inventory(string newName) : Inventory(newName, 0.0)
     {
          cout << "You are using: Inventory(st
     }
     Inventory(double newSize) : Inventory("T
     {
          cout << "You are using: Inventory(do
     }
     Inventory() : Inventory("Tool", 0.0)
     {
          cout << "You are using: Inventory()" << endl;
     }
private:
     string name;
     double size;
};
```

```
int main()
{
     Inventory myInventoryAr1[5];
}
```

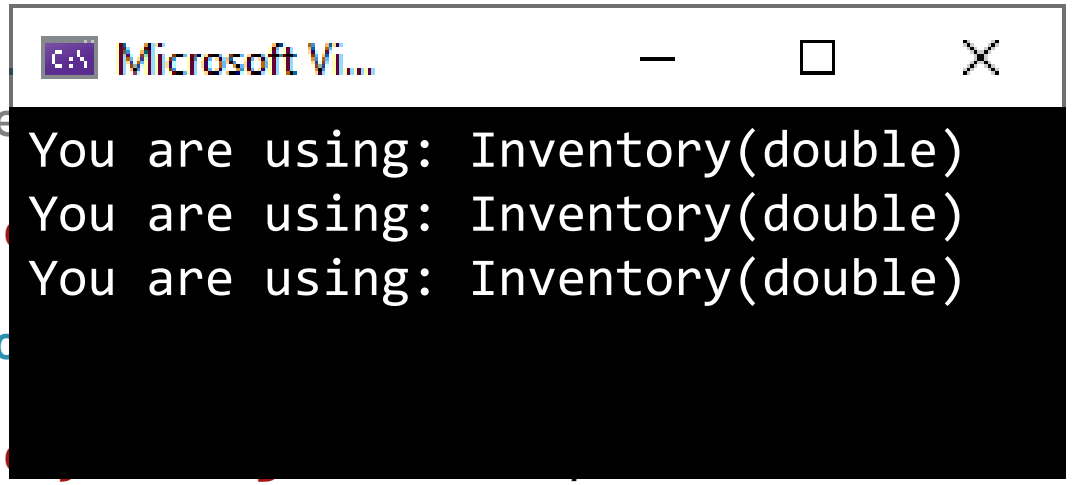
Microsoft Vi...

```
You are using: Inventory()
You are using: Inventory()
You are using: Inventory()
You are using: Inventory()
You are using: Inventory()
```

Array of Objects

```
class Inventory
{
public:
    Inventory(string newName, double ne
    {
        cout << "You are using: Invent
    }
    Inventory(string newName) : Invento
    {
        cout << "You are using: Invent
    }
    Inventory(double newSize) : Inventory("Tool", newSize)
    {
        cout << "You are using: Inventory(double)" << endl;
    }
    Inventory() : Inventory("Tool", 0.0)
    {
        cout << "You are using: Inventory()" << endl;
    }
private:
    string name;
    double size;
};
```

```
int main()
{
    Inventory myInventoryAr2[] = { 1,3,5 };
}
```

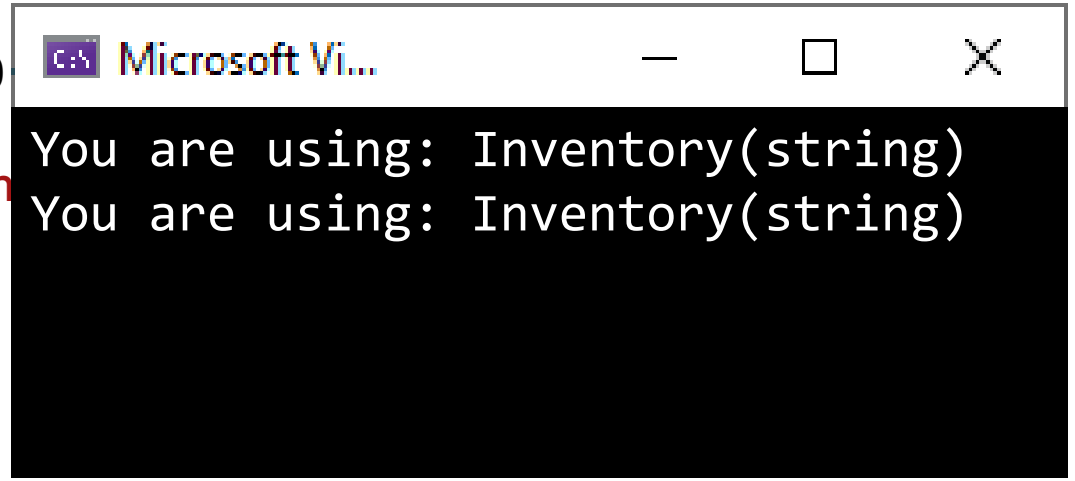


R E A D

Array of Objects

```
class Inventory
{
public:
    Inventory(string newName, double newSize) : name(newName), size(newSize)
    {
        cout << "You are using: Inven
    }
    Inventory(string newName) : Inven
    {
        cout << "You are using: Inven
    }
    Inventory(double newSize) : Inven
    {
        cout << "You are using: Inven
    }
    Inventory() : Inventory("Tool", 0
    {
        cout << "You are using: Inven
    }
private:
    string name;
    double size;
};
```

```
int main()
{
    Inventory myInventoryAr3[] =
    {
        string("Hammer"),
        string("Wrench")
    };
}
```

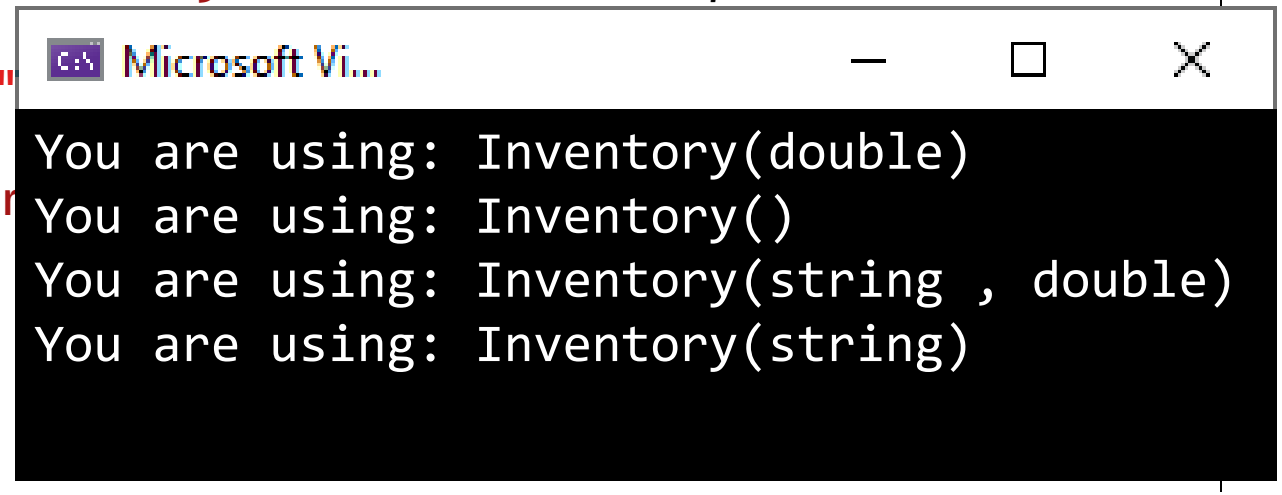


R E A D

Array of Objects

```
class Inventory
{
public:
     Inventory(string newName, double                                    ize)
     {
          cout << "You are using: Inver
     }
     Inventory(string newName) : Inven
     {
          cout << "You are using: Inver
     }
     Inventory(double newSize) : Inven
     {
          cout << "You are using: Inver
     }
     Inventory() : Inventory("Tool"
     {
          cout << "You are using: In
     }
private:
     string name;
     double size;
};
```

```
int main()
{
     Inventory myInventoryAr4[] =
     {
          1,
          Inventory(),
          Inventory("Hammer",5),
          string("Wrench")
     };
}
```

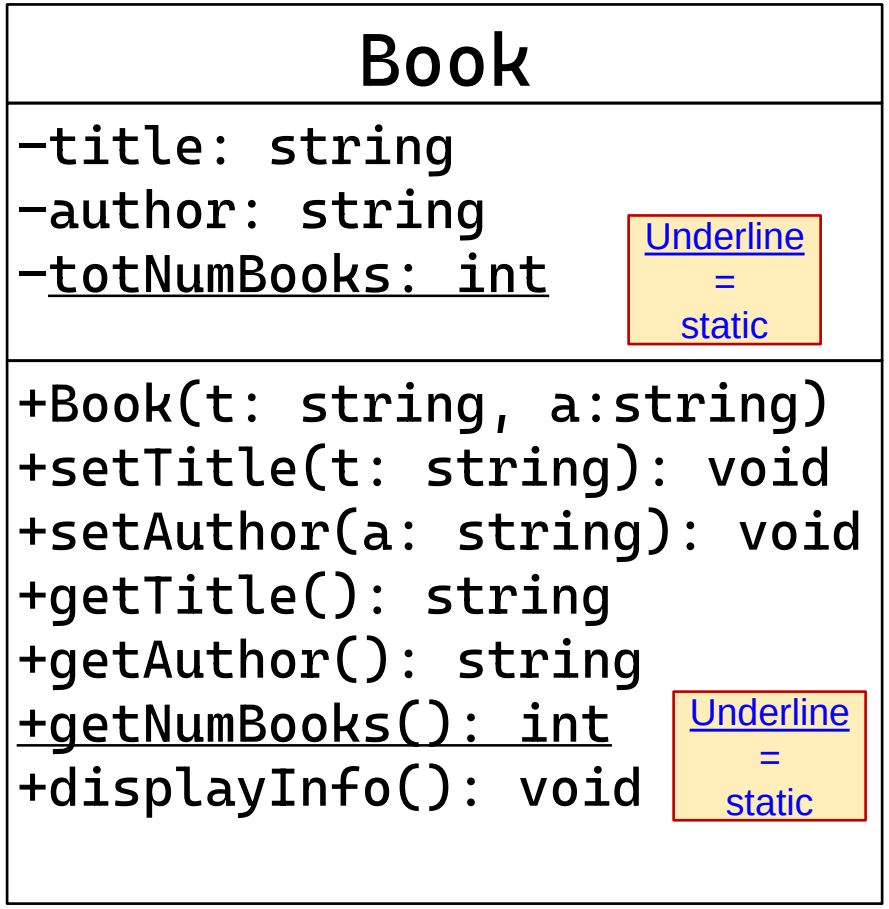


Example

Design a `'Book'` class for a library system.

- Each book should have a title and an author.
- The class needs to keep track and reports the total number of books in the collection.
- For each book, it should be possible to retrieve and update its details properly.
- Also, include a function to display book's information.
- Add a constructor for initialization.

Implement and test your class



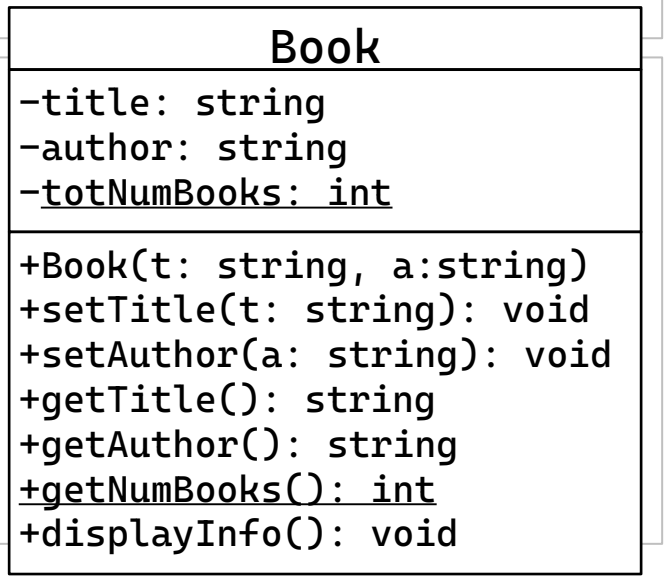
Example

```cpp
// Book.h
#pragma once

class Book
{
private:
    string title, author;
    static int bookCount; // Static member

public:
    // Constructor
    Book(string t, string a);

    // Getters
    string getTitle() const { return title; }
    string getAuthor() const { return author; }

    // Setters
    void setTitle(const string& t) { title = t; }
    void setAuthor(const string& a) { author = a; }

    // Const function to display book details
    void displayDetails() const;

    // Static member function to get total book count
    static int getTotalBookCount() { return bookCount; }
};
```

Inline functions

```cpp
// Book.cpp

int Book::bookCount = 0; // Initialize static member

Book::Book(string t, string a)
{
    title = t;
    author = a;

    bookCount++;
}

void Book::displayDetails() const
{
    cout << title << ", " << author << endl;
}
```

```cpp
int main()
{



}
```



Example

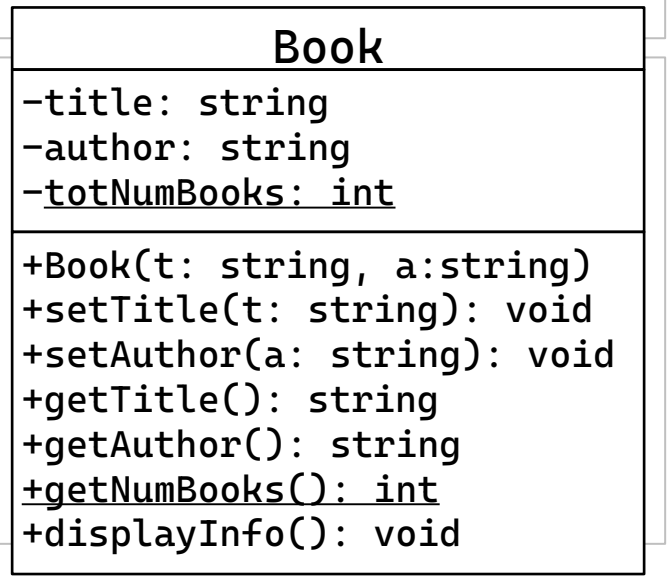
```cpp
// Book.h
#pragma once

class Book
{
private:
    string title, author;
    static int bookCount; // Static member

public:
    // Constructor
    Book(string t, string a);

    // Getters
    string getTitle() const { return title; }
    string getAuthor() const { return author; }

    // Setters
    void setTitle(const string& t) { title = t; }
    void setAuthor(const string& a) { author = a; }

    // Const function to display book details
    void displayDetails() const;

    // Static member function to get total book count
    static int getTotalBookCount() { return bookCount; }
};
```

Inline functions

```cpp
// Book.cpp

int Book::bookCount = 0; // Initialize static member

Book::Book(string t, string a)
{
    title = t;
    author = a;

    bookCount++;
}

inline void Book::displayDetails() const
{
    cout << title << ", " << author << endl;
}
```

"Inline" function

```cpp
int main()
{



}
```



Example

```cpp
// Book.h
#pragma once

class Book
{
private:
    string title, author;
    static int bookCount; // Static member

public:
    // Constructor
    Book(string t, string a);

    // Getters
    string getTitle() const { return title; }
    string getAuthor() const { return author; }

    // Setters
    void setTitle(const string& t) { title = t; }
    void setAuthor(const string& a) { author = a; }

    // Const function to display book details
    void displayDetails() const;

    // Static member function to get total book count
    static int getTotalBookCount() { return bookCount; }
};
```

Inline functions

```cpp
// Book.cpp

int Book::bookCount = 0; // Initialize static member

Book::Book(string t, string a)
{
    title = t;
    author = a;

    bookCount++;
}

inline void Book::displayDetails() const
{
    cout << title << ", " << author << endl;
}
```

"Inline" function

```cpp
int main()
{
    Book book1("Age of Science", "Ahmed Zewail");
    Book book2("The Book of Healing", "Ibn Sina");

    book1.displayDetails();
    book2.displayDetails();

    cout << "Total books: "
         << Book::getTotalBookCount() << endl;
}
```

Example

```h
// Book.h
#pragma once

class Book
{
private:
    string title, author;
    static int bookCount; // Static member

public:
    // Constructor
    Book(string t, string a);

    // Getters
    string getTitle() const { return title; }
    string getAuthor() const { return author; }

    // Setters
    void setTitle(const string& t) { title = t; }
    void setAuthor(const string& a) { author = a; }

    // Const function to display book details
    void displayDetails() const;

    // Static member function to get total book count
    static int getTotalBookCount() { return bookCount; }
};
```

Inline functions

```cpp
// Book.cpp

int Book::bookCount = 0; // Initialize static member

Book::Book(string t, string a)
{
    title = t;
    author = a;

    bookCount++;
}

inline void Book::displayDetails() const
{
    cout << title << ", " << author << endl;
}
```

"Inline" function

```
Microsoft Vi...
Age of Science, Ahmed Zewail
The Book of Healing, Ibn Sina
Total books: 2
```

```cpp
    cout << "Total books: "
         << Book::getTotalBookCount() << endl;
}
```

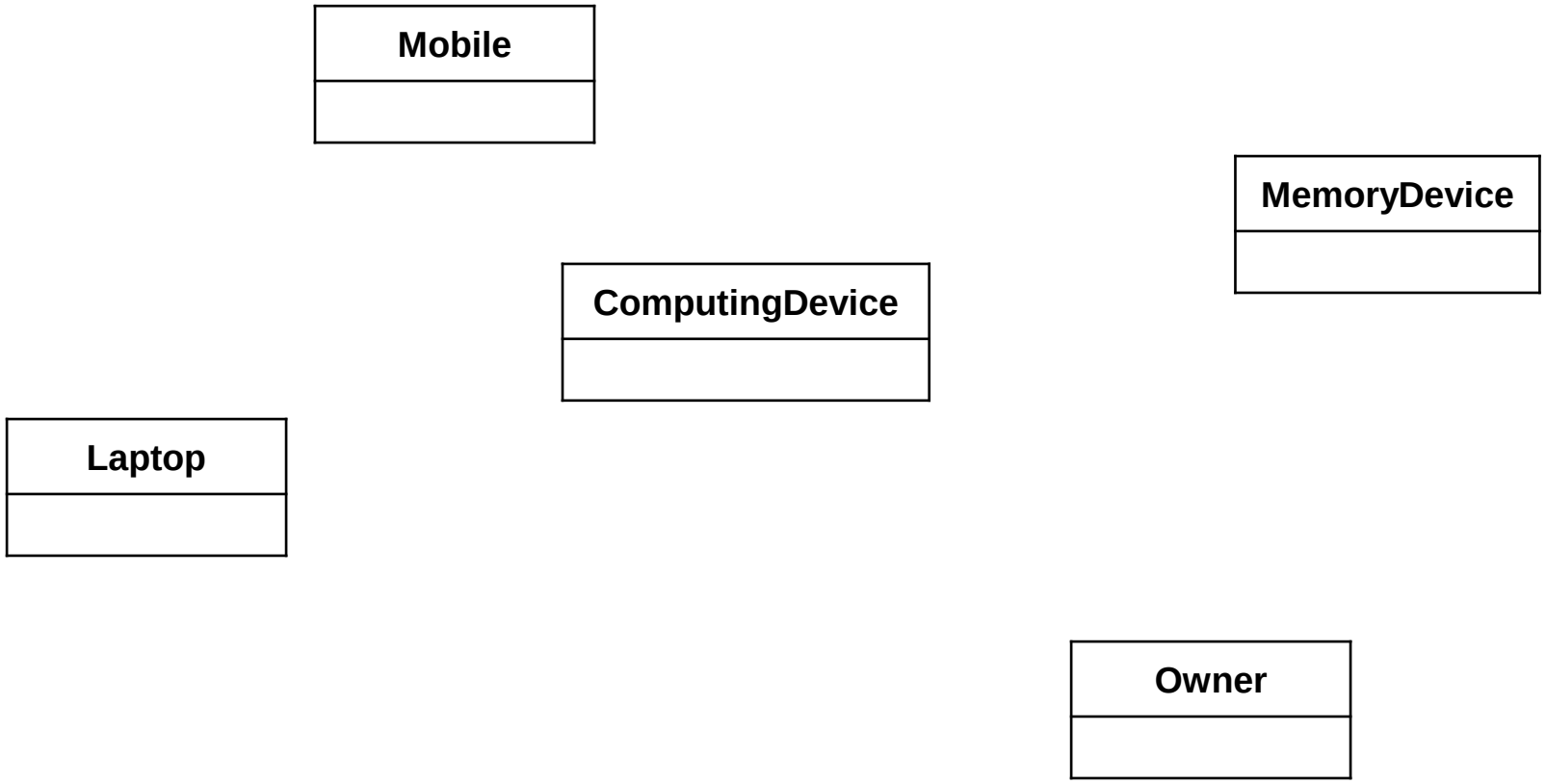
Introduction to Designing an Object-Oriented Solution

Designing a Good Class

- **Single Responsibility** – one job / responsibility per class.
- **Clarity** – use clear and descriptive naming.
- **Small / Focused Methods** – do one thing well.
- **Encapsulation** – hide internal details.
- **Abstraction** – provide simple / intuitive interface.
- **Low Coupling** – operations are independent.
- **Completeness** – provide all the necessary operations.
- **Consistency** – follow established conventions / patterns.
- **No Magic Numbers** – use named constants.

Well-designed, easy to use, and efficient

Class Relationships

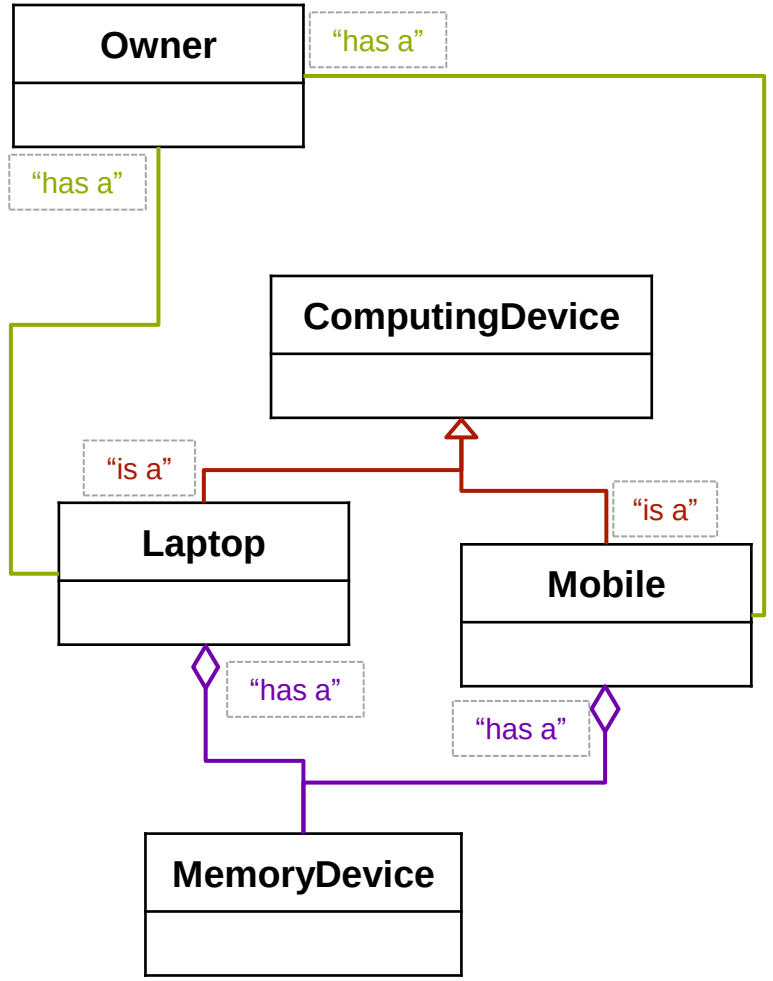


Class Relationships

- **Aggregation** / **Composition**: a Memory Device is a part of both Laptop and Mobile

- **Inheritance** / **Generalization**: a laptop is a kind of Computing Device

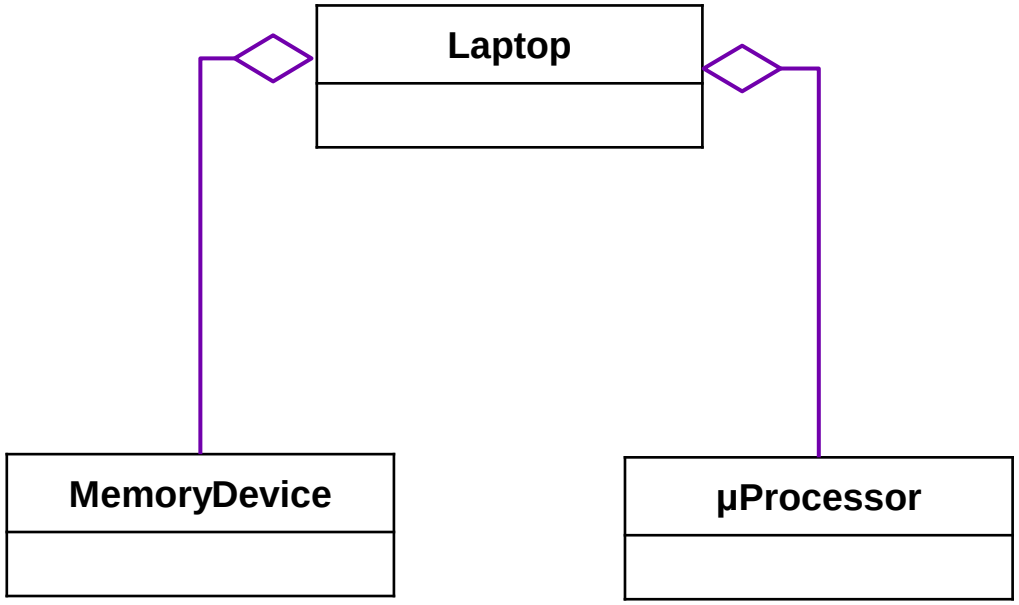
- **Association**: owners use Computing Devices, Computing Devices serve owners



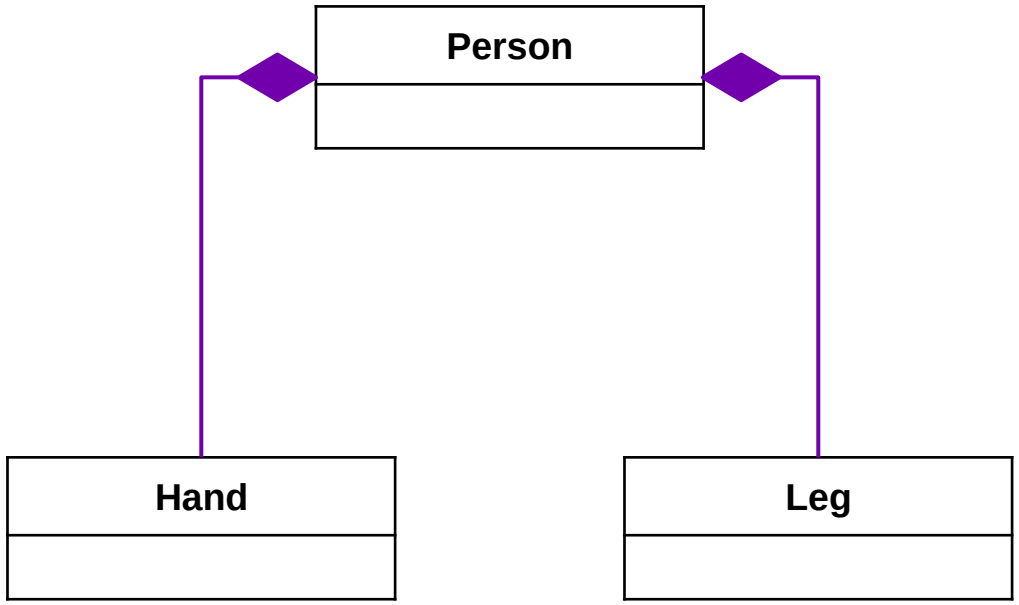
Class Relationships: Aggregation / Composition

"has a"

Aggregation

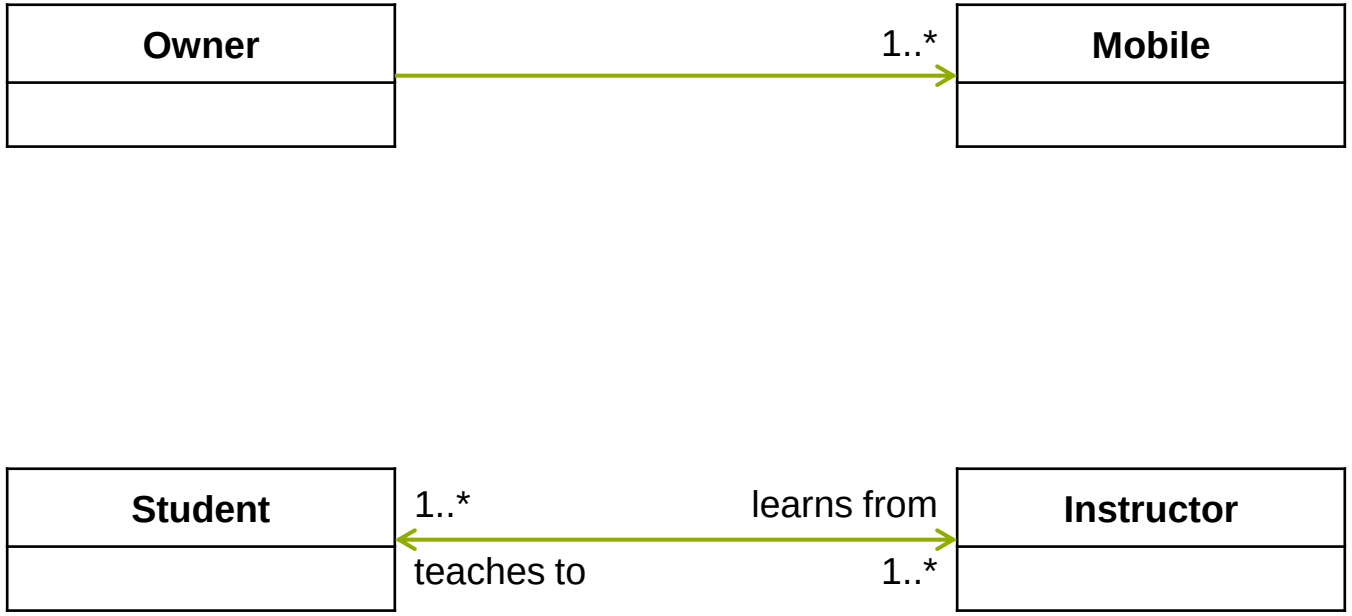


Composition



Class Relationships: Association

"has a"

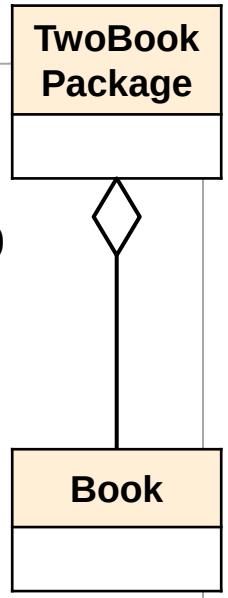


Know about each others (Provide services)

Class Relationships: Aggregation / Composition

Aggregation example

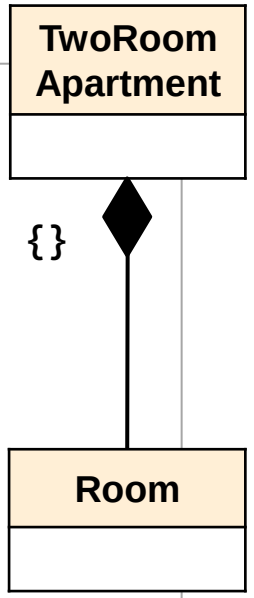
```
class Book
{
private: string title, author;
public: Book(string t = "", string a = "")
      : title(t), author(a) {}
};

class TwoBookPackage
{
public: Book &book1, &book2;
   TwoBookPackage(Book& b1, Book& b2)
      : book1(b1), book2(b2) {}
};

void testAggregation()
{
   Book book1("Age of Science", "A Zewail");
   Book book2("The Book of Healing","Ibn Sina");
   TwoBookPackage package(book1, book2);
}
```



Books exist independently of the TwoBookPackage

Composition example

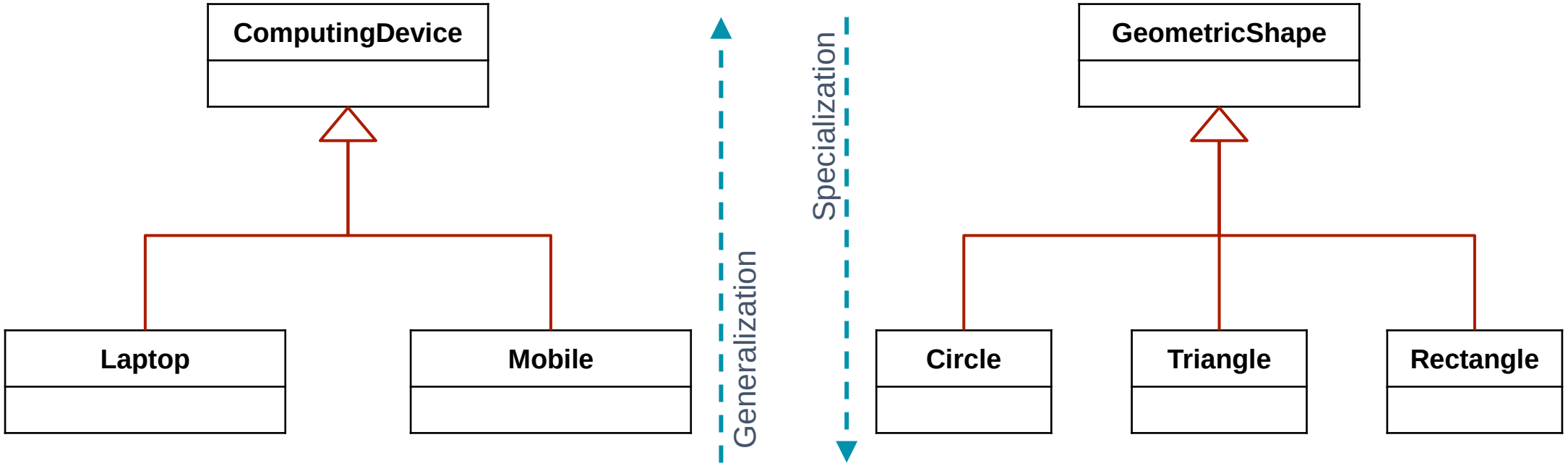
```
class Room
{
private: string name;
public: Room(string n = "Room") : name(n) {}
};

class TwoRoomApartment
{
public: Room room1, room2;
   TwoRoomApartment(string n1, string n2)
      : room1(n1), room2(n2)
   {
   }
};

void testComposition()
{
   TwoRoomApartment apartment(
      "Living Room", "Bedroom");
}
```



Rooms do not exist outside the apartment

Class Relationships: Inheritance / Generalization

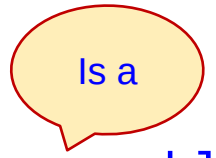
"is a"



Class Relationships: Inheritance / Generalization

```
int main()
{
	GeometricObject  o;
	cout << o.color << endl;

	Circle c1;
	cout << c1.color << endl;
	cout << c1.radius << endl;

	Square s1;
	cout << s1.color << endl;
	cout << s1.length << endl;
}
```

```
class GeometricObject
{
public : string color = "red";
};

class Circle : public GeometricObject
{
public: double radius = 2;
};

class Square : public GeometricObject
{
public: double length = 5;
};
```



Class Relationships: Inheritance / Generalization

```
class GeometricObject { ... }

class Circle : public GeometricObject { ... }

void dispGeometricObjectData(const GeometricObject& geom)
{
    cout << geom.getColor();
}

int main()
{
    GeometricObject geomObj;
    dispGeometricObjectData (geomObj);
    Circle circle;
    dispGeometricObjectData (circle);  ✅✅✅
}
```